

TOPICAL REVIEW

Balancing Performance and Cost—FPGA-Based CNN Accelerators for Edge Computing: Status Quo, Key Challenges, and Prospective Innovations

JOHN S. FATA¹, (Member, IEEE), WAFI M. ELMANNAI¹, (Member, IEEE),
AND KHALED ELLEITHY², (Senior Member, IEEE)

¹Department of Electrical and Computer Engineering, Manhattan University, Riverdale, NY 10471, USA

²Department of Computer Science, University of Bridgeport, Bridgeport, CT 06604, USA

Corresponding author: Wafi M. Elmannai (wafi.elmannai@manhattan.edu)

ABSTRACT The development of AI accelerators for real-time edge computing applications is a robust and ever-growing field that offers countless improvements upon classical CPU, GPU, or ASIC-based designs. In 2023, the edge AI accelerator market was valued at nearly \$6 billion with a compound annual growth rate (CAGR) of 30.4% over the next 6 years. However, there are currently many limitations present in the designs that have been proposed. This paper presents a comprehensive overview of FPGA-based CNN accelerators, their architectural innovations, real-world deployments, and optimization techniques. These accelerators are highly applicable for countless edge computing applications including smart surveillance systems, autonomous vehicles, traffic monitoring systems, and drone applications. Through comparative analysis, we extract key trade-offs between power consumption, throughput, hardware resource utilization, and system cost. This work not only covers architectural design strategies but also introduces a scoring framework for low-cost designs. Based on how each low-cost system performs in a specific performance field, a final score is determined. These scores allow us to identify key performance trends amongst low-cost designs, proposing solutions for the future development of such systems. Overall, our findings show that dynamically reconfigurable FPGA architectures offer appealing adaptability and performance for edge computing, at the expense of increased complexity and power consumption. Alternatively, static accelerators exhibit great potential when optimized for power and memory efficiency, at the cost of throughput. Key design considerations and future directions, addressing the prominent tradeoff between performance, efficiency, and scalability in edge AI accelerator development are discussed in the paper. We conclude that no current design fully satisfies every edge computing constraint, highlighting the need for hybrid solutions that balance reconfigurability, power, and performance at a reduced cost.

INDEX TERMS AI-accelerator, CNN model, edge computing, FPGA-based acceleration, FPGA, object detection, parallel computing, YOLO.

I. INTRODUCTION

Convolutional Neural Networks (CNNs) are an exceptional choice for computer vision applications such as object detection given their strong ability to perform video or image

processing. Due to the increasing computational complexity of current deep learning models, traditional microcontrollers are not sufficient for carrying out these tasks [1]. With the parallel processing capabilities provided using FPGAs and their ability to achieve low power consumption, low latency, and reconfigurability, they are a great choice for accelerating the speed and improving the performance of CNNs. There

The associate editor coordinating the review of this manuscript and approving it for publication was Mario Donato Marino¹.

have been many efforts in recent years to design FPGA-based CNN accelerators to take advantage of the parallel computing capabilities of FPGAs, optimizing hardware workloads to improve resource utilization, and dynamically reconfiguring FPGA computational units to achieve maximum efficiency. Increased resource integration such as on-chip memory and computational units can be combined into a single chip to improve the performance and energy efficiency of the FPGA [2]. Better hardware optimizations are an avenue to be explored including advancements in high-level synthesis tools or designing an automated hardware generation system applicable to deep learning models. When leveraging FPGAs for deep learning (DL) tasks, there is an evident tradeoff between the various performance metrics that can be achieved.

Real-time data processing has seen a massive increase in use in recent years particularly for edge computing applications [3]. This has resulted in the need for meticulously crafted devices best suited for efficient use of available hardware resources and accuracy. Edge computing is highly desirable due to the real-time processing of data locally on the device, rather than sending the data to external sources for computation [3]. This local computation reduces high-latency and potential cybersecurity threats, as opposed to cloud computing where data is transmitted to an external server. FPGAs are an excellent choice for edge AI applications given their ability to achieve low power consumption while offering low-latency processing with high computational performance. Potential FPGA-Based edge computing applications include smart surveillance systems, traffic monitoring, navigation systems for the visually impaired, and autonomous vehicle subsystems such as collision avoidance or traffic light detection [1], [2], [4], [5], [6]. Each of these potential edge computing applications operate in resource-constrained environments, meaning the device is typically lightweight, battery operated, requires long operation time, and is part of a larger system. Therefore, the device utilized for AI computation should consume minimal power, while still maintaining efficient and accurate performance. Traditional CPU or GPU based implementations for embedded systems can offer high performance, but struggle with power consumption. This is because such processors are multi-purpose and operate at very high clock speeds. While a high clock can attain good performance in terms of computational throughput, it drastically increases power consumption, thus decreasing efficiency. The parallel computing capabilities of FPGAs allow us to leverage the system for high throughput, while minimizing power consumption. FPGA-Based accelerators are single purposed, meaning the architecture implemented in the FPGAs programmable logic (PL) has a specific task (accelerating CNN operations). Comparing this behavior to that of a CPU or GPU, which are multipurpose processors, are engineered for running a variety of tasks. This allows an FPGA-based design to run on a lower clock cycle, which limits power consumption while still achieving high throughput. In 2023,

the edge AI accelerator market was valued at nearly \$6 billion with a compound annual growth rate (CAGR) of 30.4% over the next 6 years [7]. The high current market value and CAGR indicate a significant need for AI accelerators in the present day, as well as the foreseeable future. Of course, there are many challenges that are faced when implementing edge AI accelerators on FPGAs. The most significant being limited hardware resource availability, including On-Chip memory [2]. This is especially the case for lower cost FPGAs. When an accelerator is implemented on an FPGA, the design is synthesized into a bitstream before it gets deployed in the FPGAs PL. The PL configures your design utilizing its LookUp Tables (LUTs) to implement logic functions, Flip-flops (FFs) to store data, and Digital Signal Processors (DSPs) to handle heavy mathematical operations. If a low-cost FPGA is utilized, fewer of these hardware resources are available as opposed to a higher cost system. This could limit the potential performance of your design. Therefore, it is crucial to optimize the architecture of the accelerator to account for these limitations, especially for low-cost systems.

In this paper, we will be exploring various approaches which leverage the parallel computing capabilities of FPGAs to develop CNN accelerators for image or video processing. By taking a hardware solution to address software-related challenges, researchers can fine-tune the system for specific applications, achieving impressive performance metrics and accurate results. Figure 1 highlights five main categories for optimization directions and corresponding techniques that can be applied for each. The approaches reviewed in this study discuss each of these techniques and how they applied it to their system. It is worth noting that techniques across multiple optimization directions can, and should, be applied for a given system. These optimization techniques are not mutual exclusive, as they improve a very specific aspect of the system. When designing an FPGA-Based accelerator, ideally, the system should achieve balanced performance metrics. This results in a robust and efficient design. Each optimization direction addresses a specific consideration found in ideal edge computing platform deployment. These considerations include minimizing power consumption, minimizing latency, compressing the size of the deployed DL model, and attaining high efficiency, throughput, and accuracy [1], [2], [3].

Many studies reviewed in this paper have utilized the techniques portrayed in Figure 1. The following paragraphs of this section highlight the following items: a brief explanation for each optimization direction and their importance, an explanation for applied techniques, additionally, examples of studies reviewed in this paper that applied these techniques are mentioned:

1. **Quantization & Compression:** Deep Learning models (especially for video & image processing) can be very large in terms of storage requirements. Given the limited availability of FPGA resources, it is optimal to compress, or reduce, the size of the deployed model.
 - INT8 Quantization is being progressively utilized to reduce the activation and weight bit-widths for

power-saving storage and improved computation speed. Al Amin et al. [5], Zhang et al. [8], and Kim et al. [9] quantized 32-bit floating-point models to 8-bit fixed-point representations, reducing memory bandwidth and power at acceptable levels of precision for object detection. This is also highly beneficial for edge computing applications in general.

- Hardware Pruning removes redundant weights or filters from CNNs, which decreases the amount of multiply-accumulate (MAC) operations required. When designing an accelerator on an FPGA, the DSPs in the FPGAs PL are responsible for performing MAC operations, which effectively perform convolutions on feature maps for video/image processing applications. These MAC operations can be performed in parallel which accelerates the computational performance of the system. This leads to lower latency and makes it possible to deploy CNN models on lower-cost FPGAs with fewer DSPs. Syed et al. [10], Irmak et al. [11], Sanchez et al. [12], and Afif et al. [6] applied hardware pruning in their design to obtain these benefits.
- Fixed-Point Optimization lowers the numerical representation complexity of weights in a neural network for faster computation. This concept differs from INT8 quantization, given that Fixed-Point optimization allows weights to be converted to a specified precision (2-bit, 6-bit, 10-bit, 12-bit, etc.). Cho and Kim [13] replaced floating-point operations with fixed-point to conserve latency and DSP utilization, showing that accuracy can be maintained at close to original levels in spite of quantization.
- LUT-Based Approximate MAC operations can be employed for MAC operations offloading from DSPs. The benefit for such a technique is that parallel computations can still be performed without the need for high DSP use, which are limited in availability and consume more power on the device. Cho and Kim [13] employed LUTs to realize approximate multipliers for low-precision convolution to reduce power on resource-constrained devices.

2. **Memory Utilization:** FPGAs are comprised of two main types of memory, On-Chip and Off-Chip memory. On-Chip memory is embedded directly within the FPGAs fabric (PL), where the deployed CNN model also resides. On-Chip memory utilization results in lower latency for data retrieval and lower power consumption but is more limited in capacity. While available Off-Chip memory is more abundant, its utilization results in lower performance metrics. Therefore, Off-Chip memory access should be avoided.
- High On-Chip BRAM/URAM Utilization is given priority in [9] and [14], where the authors attempted to utilize on-chip resources to their maximum (including memory) so that off-chip memory access is minimized.

This directly affects latency and energy efficiency in a positive manner.

- Memory Reuse between Layers is employed in [15], where dynamic reuse of feature maps between convolutional layers is facilitated by an elastic buffer. This removes redundant memory access which enhances throughput.
 - Off-Chip Memory Avoidance is an appropriate design practice for most designs. While there may be high off-chip memory available, it is physically far away from the PL which causes higher data read/write times and power consumption. Nguyen et al. accommodated the entire YOLO model in BRAM to prevent the usage of external DRAM for faster data access in real time [16].
 - AXI-DMA Optimization involves optimizing direct memory access paths between components transferring data to each other amongst the PL and processing system (PS). This minimizes transfer overhead and improves throughput in real-time inference. Shi et al. implemented such an approach [14].
3. **Reconfigurability:** A main advantage for FPGAs is the ability to reconfigure hardware in the PL in real time. This can allow the system to adjust to dynamic workloads on the fly, resulting in a more efficient system.
 - Dynamic Partial Reconfiguration (DPR) is a method by which portions of the FPGAs PL can be reconfigured while the FPGA is running. This differs from a fully reconfigurable design, in which the entire PL can be reconfigured in real time. DPR offers the advantage of reconfigurable hardware, but without such high overhead and complexity as opposed to complete reconfiguration. DPR provides flexibility and saves static hardware costs for multi-task or dynamic workloads. Irmak et al. [11], Shi et al. [14] and Hale [17], all introduced a DPR-based accelerator which allows the system to reconfigure PL hardware in real time. This allows for increased flexibility, and even enhanced resource utilization.
 - Mode Switching in [10] allows the system to switch between high-performance, de-stress, or fault-tolerant modes depending on runtime conditions. This dynamic flexibility is beneficial in energy-limited or mission-critical systems.
 - Elastic On-Chip Buffering, employed in [15], dynamically resizes internal buffer allocations in accordance with the size and depth of each CNN layer. This helps in efficient utilization of available memory in real time.
 - Runtime Reconfigurable Compute Cores were illustrated in [17], where they isolate static and reconfigurable modules to balance efficiency and flexibility, paying particular attention to modular CNN processing pipelines.
4. **Energy Efficiency:** For edge computing applications, low power consumption is vital. Ensuring that the

system is performing high, while consuming minimal power is ideal.

- Dynamic Voltage Frequency Scaling (DVFS) is a power reduction technique that dynamically adjusts the clock frequency and voltage of the system during runtime. This offers further management of the power/performance trade-off. An increased clock speed promotes higher throughput, but results in increased power consumption. A lower clock speed reduces throughput as well as power consumption. With DVFS, the clock speed and voltage levels of the system can be changed depending on the complexity requirements of the application at a given moment in time. Lin et al. [18] implemented a system with DVFS to attain these benefits.
 - Integer Arithmetic-Only Computation, introduced by Kim et al. in [9], eliminated floating-point units and kept integer mathematics only. The method aggressively reduced logic and power at the cost of functional loss to low-precision applications.
 - Clock Gating, implemented in [19], turns off inactive parts of the hardware during idleness. This fine control over switching activity conserved dynamic power and improved overall energy efficiency for edge computing applications.
 - Hardware-Aware Model Scaling refers to the reduction of DL models' parameters to more adequately fit in the FPGAs resources. Nguyen et al. [16] reduced a YOLO model to exclusively fit into on-chip memory, minimizing power and promoting higher throughput. This directly correlates to a highly efficient system.
- 5. Throughput & Parallelism:** Throughput is the performance metric that signifies how computationally well the system performs. It is typically measured in operations performed per second (OPS), or frames process per second (FPS) for video/image processing applications. By having high hardware parallelism (accelerated computing), you get higher throughput.
- Layer-Wise Pipelining is taken by Nguyen et al. [16] so that each CNN layer directly sends the output to the next layer without buffering delay, which increases the inference speed (throughput) while minimizing memory read/writes.
 - Multicore CNN Engines are emphasized in [15], where inter-layer parallel computation of CNN was implemented by a tile-grained multicore processor with up to 1503 GOP/s throughput. A multicore CNN engine promotes very high hardware parallelism, allowing many convolutional layer filters to be applied to a single feature map simultaneously.
 - Streaming Architecture is demonstrated in [8], where data are streamed continuously from a streaming convolution engine in order to reduce memory read/write latency as well as switch-over latency for layers. This reduced latency time for memory access promotes higher throughput.

- Resource Tiling & Reuse are demonstrated in [20], where reusable computation kernels over tiled image partitions achieve maximum LUT usage as well as DSP utilization within spatial domains, exhibiting high area efficiency. This results in very high throughput due to the reuse of computation kernels in the design.

II. SIGNIFICANT FPGA ACCELERATORS FOR CNN VIDEO AND IMAGE PROCESSING

A. RELATED LITERATURE SURVEYS

We reviewed several survey papers that discussed FPGA applications in various domains. The following review papers were the most related articles: Kai Zeng et al. reviewed and compared FPGAs with most common hardware accelerators. Authors in addition summarized several object detection algorithms. This research addressed two main components in their literature survey: why to choose FPGAs, what are the design methods and the design goals of FPGA [1]. Haobo Ye explored in [2] the CNN principles, FPGA advantages for accelerating CNN algorithms. The author discussed four FPGA based CNN models in terms of reconfigurability, energy efficiency and parameterizable. In addition, the authors highlighted the challenges of the four models from the use of the hardware resource, mapping the algorithms, and balancing between the efficiency and flexibility. Although the paper is very interesting, the authors explored very limited designs and highlighted few challenges and recommendations. Similarly, the authors of [4] briefly explored several designs of deep learning algorithms in FPGAs. The authors compared several FPGAs based deep learning architectures. The implementation analysis was done in terms of the energy efficiency and performance to determine the best implementation architecture.

On the other hand, our paper provides a complete overview of 20 different FPGA-Based CNN accelerators including their main approach, advantages, performance parameters, most significant limitations, and suggested future work. A comparison table of each system's performance parameters is presented, allowing researchers to see immediately the pros and cons of methods discussed in this work. This is superior to other reviews in literature that lack such aids [1], [2], [4]. Further, we perform an in-depth analysis of low-cost designs and determine dominant Figure of Merits (FOMs) of real-time edge applications and infer trends in such systems. We enlighten the reader with insights into likely solutions and future research derived from the inferred tradeoffs toward improving low-cost designs. We notably emphasize future research on low-cost designs for edge computing deployment based on the visible tradeoff inferred in the analysis. It is notable that edge applications are typically mass-produced by companies, where low-cost devices are in great demand [3]. Especially if they are high-performance, low-power devices, tuned to the appropriate application. This is another significant improvement of this literature review over the others. Other reviews don't mention the importance

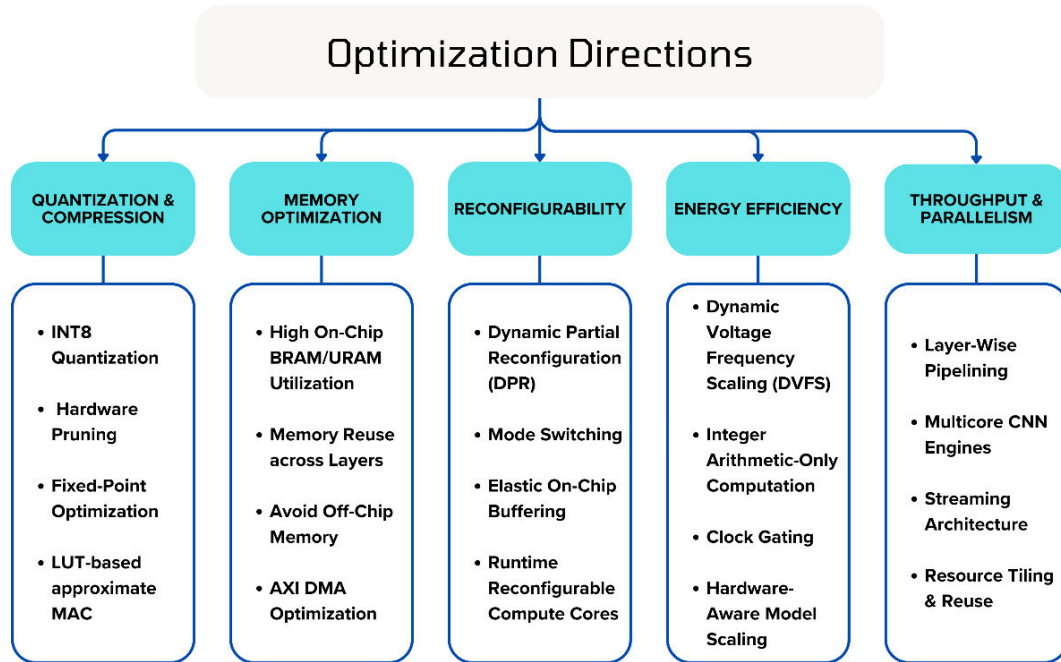


FIGURE 1. FPGA CNN accelerator optimization directions & corresponding techniques.

of low-cost FPGA accelerator designs and emphasize edge computing deployment directly [1], [2], [4].

Additionally, although earlier FPGA surveys [1], [2], [4] focused on high-level architectural taxonomies, reconfigurable hardware abstractions, or application-specific use cases, they failed to introduce a quantitative scoring model or emphasize low-cost and low-power designs that are deployable on the edge. This paper introduces a comparative scoring method on five principal performance indicators, performs a focus analysis on sub-\$1,000 FPGA platforms, and aggregates findings in terms of architectural, cost, and resource dimensions to guide future low-power AI accelerator designs.

B. CLASSIFICATION BY DESIGN INTENT: ARCHITECTURAL, APPLICATION-SPECIFIC, AND ENERGY-AWARE APPROACHES

FPGA-based accelerators for CNNs are extremely versatile in design style due to the programmable nature of logic and the rich set of challenges offered by edge computing environments. As a result, researchers have proposed a broad range of solutions targeting different accelerator design dimensions ranging from architectural novelty and application-specific customization to aggressive power saving techniques. In this sub-section, we categorize the surveyed literature into three general categories based on the overall design focus of the authors: (1) novel hardware architectures, (2) real-world embedded system applications, and (3) energy-conscious system designs. Such a theme-based organization facilitates an end-to-end view of how different priorities such as fault-tolerance, reconfigurability of on-chip resources, or application-specific deployment affect architecture selection and system performance. Classification by design goals provides insightful guidance into directions of

current innovation and suggests the implementation motivations underlying design. Each paper is evaluated not only on its contribution to method, but on its performance metrics, hardware trade-off, and asserted limitations. Such classification helps to make visible emerging trends and chronic constraints in the field.

To aid future research and system design, we provide a thorough analysis of the tradeoffs of each design in power, performance, resource utilization, and scalability. This thematic grouping highlights how the varying design priorities at the start, whether architectural novelty, application specificity, or energy conservation, can lead to divergent results for basic performance metrics. In Section III, we reclassify the designs surveyed based on functional implementation goals including their degree of reconfigurability, power consumption, cost-effectiveness, and resource utilization. This hierarchical classification allows for a more detailed understanding of the ways in which design choices directly impact real-world constraints, permitting deeper insight into the most interesting tradeoffs and most promising directions for future FPGA-based CNN accelerators.

To further assist interpretation of each work in this section, a comprehensive comparison chart is included in Section III (see Table 2). This chart provides insightful summaries of each work that easily allows you to directly compare each system seamlessly.

C. NOVEL HARDWARE ARCHITECTURES

1) DYNAMICALLY RECONFIGURABLE COLUMN STREAMING CNN

Lin et al. [18] developed a dynamically reconfigurable column streaming-based CNN for edge computing applications.

TABLE 1. Evaluation of reviewed systems based on addition factors and the limitations of each system.

System Name	Power Consumption	Energy Efficiency Approach	Throughput	Type of NN	Accuracy/Efficiency	Cost	Limitation	Resource/Memory Utilization
Kria KV260 FPGA-based YOLO System [5]	3.5W	Quantization	15 FPS	YOLOv3 Tiny	99% Accuracy	\$250	Limited On-Chip memory available	Off-Chip + On-Chip memory Used
Indoor Object Detection Accelerator [6]	5W	Hardware Pruning Quantization	17 FPS	CSPNet + EfficientNet v2	80.04% mAP	~\$3,300	High computational complexity	N/A
YOLOv2 FPGA-based System [8]	5.96W	Quantization Layer fusion	111.5 GOP/s	YOLOv2	69.32% mAP 18.71 GOPs/W	~\$500	External memory bandwidth	369 BRAMs Utilized Optimized DDR usage
Resource Efficient FPGA-Based CNN Model [9]	2.3W	Quantization	10 FPS	SqueezeNet	86.58% accuracy	~\$3,000	High Cost Low FPS	Utilizes 61 BRAMs, 49 DSPs
Reconfigurable Fault-Tolerant CNN [10]	0.393 - 0.66W (based on mode used)	Hardware Pruning Quantization Shared Layers	108.88 GOPS (DS mode) 326.6 GOPS (FT mode) 0.979 TOPS (HP mode)	CNN	90% Accuracy	\$15,000	Limited DSP usage Memory usage varies based on mode ran	N/A
Dynamically Reconfigurable NN Accelerator on FPGA [11]	2.039W	DPR Hardware Pruning Resource Sharing	40.71 GOPS	Hybrid CNN + SNN	99.3% Accuracy	\$300-\$500	DPR overhead High Complexity High Latency	N/A
FPGA Smart Camera for Segmentation [12]	N/A	Hardware Pruning Quantization	33.63 FPS (multi-threaded) 15.77 FPS (single-threaded)	U-Net (modified, pruned, and quantized)	MCC of 91.2% F1 Score of 91.5% Hafiane of 87%	\$300	Drop in performance during Inference	N/A
LeNet-5 CNN Model [13]	N/A	Fixed-point Quantization Approximate MAC operations	N/A	CNN	<1% accuracy loss after Quantization	~\$500	Memory bottleneck	66% reduced memory compared to floating-point mode
EDRCA Accelerator [14]	5.520 W	Dynamic Reconfiguration	75.1929 GOPS	YOLOv2-Tiny	13.6219 GOPS/W	~\$250	Resource limitations due to hardware Requires optimization	100% URAM + BRAM Utilization
Reconfigurable NN for Obj. Detection [15]	12.6W using MobileNetV2 15.3W using YOLOv3	Elastic on-chip buffer Hierarchical pipelining Reconfigurable Design	809–1503 GOPS (Based on model used)	Multiple Tested: RetinaNet-ResNet-50, MobileNetV2-SSDLite, YOLOv3	For Quantized Models → accuracy loss less than 2 mAP Computational Efficiency up to 0.79 GOPS/DSP	~\$3,500	Requires off-chip memory High Power Consumption High Cost	High On-Chip memory usage

TABLE 1. (Continued.) Evaluation of reviewed systems based on addition factors and the limitations of each system.

High-Throughput & Power-Efficient FPGA Implementation of YOLO CNN [16]	18.29W	Intermediate data reuse Exclusive On-Chip utilization Low-Bit Quantization	1.877 TOP/s	YOLOv2	mAP of 64.16%	\$5,000-\$10,000	High Power Consumption High Cost	100% On-Chip Memory Utilization (BRAM + URAM)
Dynamic Reconfigurable CNN Model for	2.92W	Dynamic Reconfiguration	26.2 GOP/s	Custom CNN (MNIST)	Energy efficiency of 8.97 GOPs/W	~\$250	Limited scalability for complex applications	Reduced hardware resource
Embedded Edge Computing [17]					Processing time of 0.142ms			utilization
Dynamically Reconfigurable Column Streaming CNN [18]	N/A	Quantization DVFS	N/A	CNN	N/A	N/A	High memory bandwidth requirements	N/A
Reconfigurable CNN RTL Accelerator [19]	1.44W	Clock Gating XOR-based MAC Memory partitioning	9.17 FPS	CNN (ResNet-20)	43.9 GOPs/W	\$250-\$500	Real-time processing limited Optimization needed for high-performance tasks	N/A
FPGA-Based YOLO System [20]	3.87W	Fixed-point quantization Batch-normalization folding	32 FPS on YOLOv3-Tiny 31 FPS on YOLOv4-Tiny	YOLOv3-Tiny YOLOv4-Tiny	Minimal Reduction in mAP after Quantization	~\$1,000	Resource Intensive High Cost	BRAM of 384
Accelerated FPGA-Based Parallel CNN-LSTM [25]	0.717W-1.744W (Based on SUs used)	4-Stage LSTM pipeline	N/A	Parallel CNN-LSTM	96.54% Accuracy	\$5,000	Long processing times	64.53% BRAM Utilization
YOLOv5 on Zynq-7000 SoC [26]	N/A	Efficient resource usage designed in Vivado Minimized off-chip memory	120 FPS	YOLOv5	N/A	\$1,160	High development time Low BRAM Utilization	78% LUTs, 52% FFs, 42% BRAMs, 8.4% DSPs
Eye Detection Accelerator [28]	51W	Discards unfocused images	87 FPS	YOLOv3-Tiny	100% mAP (256x256) 95.31% mAP (416x416)	~\$550	High Power Consumption	N/A
ERVSR Accelerator [29]	5.47W	Channel modulation coefficient Hidden state compression	76 FPS	RCNN	N/A	~4,000	Memory intensive	N/A
SparkNOC Lightweight Accelerator[30]	7.58W	Fully pipelined architecture	337.2 GOP/s	SparkNet	44.48 GOP/s/W	~\$4,500	Limited Reconfigurability & Parallelism	13.7% BRAM utilization 96.9% DSP utilization

A column streaming-based approach divides the input data into columns allowing data to be computed in parallel. This is one of the major advantages that an FPGA provides. This engine can be reprogrammed before each convolution operation, meaning that Lin’s approach enables dynamic reconfiguration based on the current input data [18]. This enables the most efficient utilization of the parallel computing capabilities of the FPGA, improving overall computational efficiency [2]. Similarly, Luo et al. [21] proposed a reconfigurable spatial architecture for accelerating inception neural networks. His design allows the computation units within the FPGA to be dynamically reconfigured based on the current network requirements such as computation complexity, memory bandwidth and size, and current latency/throughput [2]. This makes Luo’s architecture applicable for real-time processing requirements, which is crucial for real-time computer vision applications.

2) RECONFIGURABLE FAULT-TOLERANT CNN ACCELERATOR

Syed et al. [10] proposes a fault-tolerant CNN accelerator including reconfigurable capabilities for fault-tolerant (FT), high-performance (HP), and de-stress (DS) modes. The traditional approach of implementing a single hardware accelerator architecture has been improved by Syed by taking on a shared layers approach. This process is assisted by pruning and quantization techniques to reduce hardware resource and power consumption while implementing pattern-learning capabilities [10]. This allows CNN to perform uncorrelated tasks using a single hardware accelerator. The switching mechanism between different modes is shown in Figure 2. Based on the current needs of the system, the accelerator switches modes leveraging high performance, fault tolerance, or de-stress for the CNN. T1, T2, and T3 represent different tasks whereas S0-S5 represent the 6 different switches which determine which accelerator is most adequate for the given task.

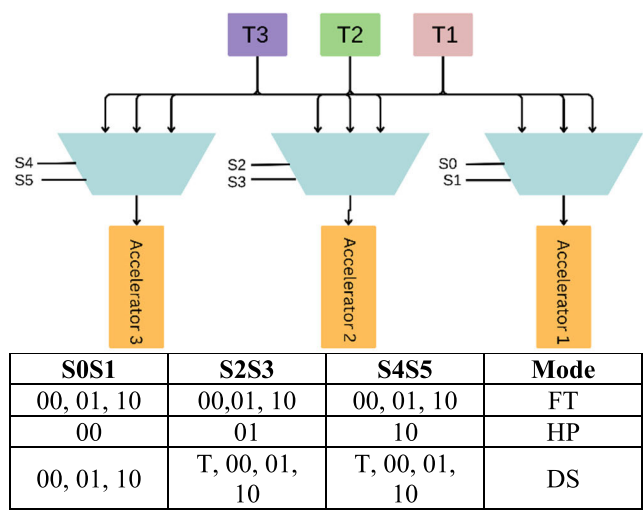


FIGURE 2. Switching mechanism for reconfigurable accelerator [10].

There has been a rise in hardware performance within hardware accelerators (FPGAs) in recent years driven by reduced transistor sizes [10]. This reduction in transistor size increases the probability of transient faults and lower circuit performance over time due to Bias Temperature Instability (BTI) [10]. These constraints have given way to Syed’s fault-tolerant methodology. While their work has addressed many current limitations with its fault-tolerance and reconfigurable capabilities, there are notable improvements that can be made. By introducing On-chip sensors for monitoring temperature and component aging, a reconfigurable AI algorithm, and reconfigurable FPGA hardware, a fully reconfigurable AI processing system to optimize performance can be developed [10]. Syed compares his work with similar FPGA and ASIC-based implementations. Compared to other FPGA-based implementations, Syed achieved better performance even in the de-stress (DS) mode where overutilization of FPGA resources is reduced. Overall, compared to similar FPGA-based designs, Syed finds that his work demonstrates competitiveness in terms of performance and energy consumption along with the added reconfigurability functionality. While ASIC-based designs have higher energy efficiency, enhanced support for reconfigurability directly addresses the dynamic environment of AI applications.

3) DYNAMICALLY PARTIAL RECONFIGURABLE NN ACCELERATOR ON FPGA

A major trend in recent years is to implement dynamically reconfigurable hardware by leveraging FPGAs to solve software problems. Dynamic Partial Reconfiguration (DPR) allows specific regions of the FPGA to be reconfigurable, while keeping other regions static. A disadvantage of such an approach expressed by other proposals is the lower throughput in comparison to static designs. Irmak et al. [11] proposes an energy-efficient DPR accelerator while maintaining throughput performance. Optimizations to balance resource usage and performance were considered along with the ability to implement different NN models on the same FPGA. The ability to deploy multiple NN models on the same device showcases this designs robustness. Figure 3 portrays the proposed hardware architecture in which different processing elements (PEs) are interconnected. These interconnections can be reconfigured during run-time allowing the type of neural network to be changed at any moment.

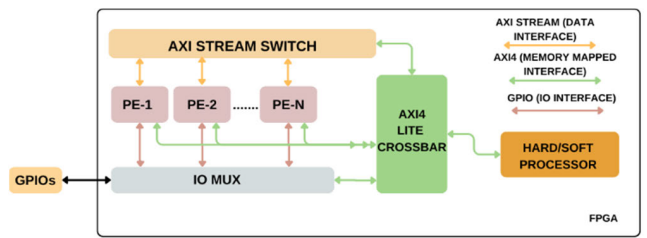


FIGURE 3. Reconfigurable NN architecture [11].

DPR reconfigures part of the FPGA while the rest of the device is still running, offering the ability to overcome the resource and power limits of the design. This enables run-time reconfiguration of selected regions of the FPGA. There have been low-power NN accelerators proposed in recent years without using DPR techniques. However, these approaches lack flexibility for specific CNN models and tend to consume more power than DPR designs. Additionally, they attain lower computing capabilities and are less applicable for edge computing applications given their reliance on energy-efficiency. Irmak achieved a 7x faster frame rate in comparison to designs that are not reconfigurable. They achieved high memory utilization with a BRAM of 58.5%, a DSP of 167, and a LUT utilization of 24,980. This system also achieved a processing time of 62us, promoting very high frame rate capabilities. There are two main advantages to using DPR architecture. Firstly, instead of training different datasets together, realizing specialized classifiers as different NN greatly increases the accuracy in each task. Secondly, by using DPR there is a great reduction in power consumption as the authors achieved an average power consumption of 1.2W during their conducted experiments [11].

4) RECONFIGURABLE NN w/MULTICORE PIPELINE FOR OBJECT DETECTION

Chang et al. [15] proposes a reconfigurable CNN processor with hierarchical pipelining and multicore reconfigurable computing. The pipelined multicore processing architecture is capable of adapting to current computational demand, as well as on-chip memory requirements of the different convolution layers. Additionally, they propose an elastic on-chip buffer by dynamically reconfiguring addresses to utilize available memory at a higher rate. The multicore architecture can be seen in Figure 4. Primary components consist of the computing engine which performs NN operations, an on-chip buffer which is dynamically configured based on feature maps, and the control unit which processes instructions.

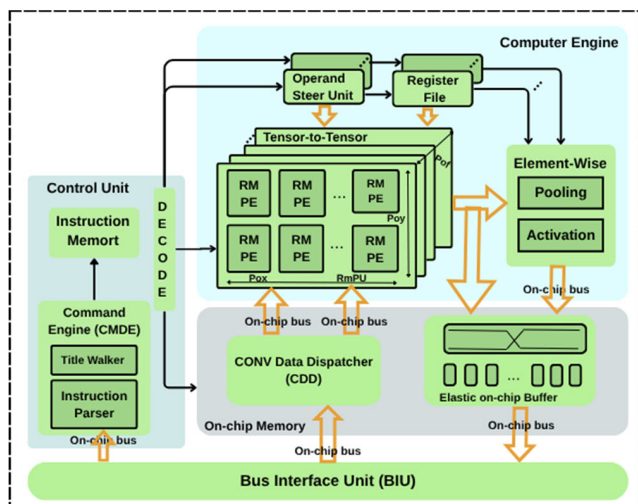


FIGURE 4. Multicore architecture [15].

This approach is necessary as most model sizes for object detection are very large which drives up the computational cost on cost-efficient computing devices. Chang claims that FPGAs cannot store all intermediate feature maps for object detection CNN models by solely relying on on-chip memory [15]. Therefore, the authors utilize a dynamically allocated multicore architecture which improves the resources allocated and computational efficiency. The object detection model was implemented on the RetinaNet-ResNet-50, MobileNetV2-SSDLite, and YOLOv3. An experimental procedure was performed to determine the performance of their design for each of the different CNN models mentioned. For each object detection model, a throughput of 1503, 1066, and 809 GOPS were achieved respectively. Additionally, computational efficiencies were measured in terms of GOPS/DSP. For each model, a computational efficiency of 0.79, 0.62, and 0.36 were achieved respectively. These results saw a significant increase in comparison to a pipelined processor designed using ResNet-152. Chang saw an improved throughput of 24% and a computational efficiency of 119%. Their future work will highlight more improvements of their proposed CNN processors based on neural architecture search (NAS).

5) DYNAMIC RECONFIGURABLE CNN ACCELERATOR FOR EMBEDDED EDGE COMPUTING

Hale [17] proposes a dynamically reconfigurable CNN accelerator, designed specifically for low-power edge computing devices. For the design, they emphasized optimization of modularity and flexibility of the system. The proposed accelerator is composed of two distinct components, the programmable logic (PL) and processing system (PS). The PL is not entirely configured to be reconfigured, rather the logic circuitry is divided into static and dynamic regions. This is referred to as Dynamic Partial Reconfiguration (DPR). The functional modules are capable of dynamically reconfiguring the convolution activation module, pooling module, fully connected module, and direct-memory access (DMA). The benefit of such an approach is the flexibility and optimization that can be achieved for specific applications including object detection and object recognition by prioritizing specific layers within the CNN model. The architecture of the proposed accelerator can be found in Figure 5.

The experimental procedure analyzed the clock frequency to be 200MHz, power-consumption of 2.92W, throughput of 26.2 GOP/s, processing time of 0.142PL/ms (programmable logic usage per millisecond), and energy efficiency of 8.97GOPS/W. Comparing these results to similar systems, Hale saw his approach see a reduction in power-consumption by up to 62.56%, reduced processing time on the PL by up to 14,195.77%, and improved energy-efficiency ratios by up to 281.7% [17]. These impressive results highlight the strength of Hale's study and its direct applicability to resource and power restricted edge computing devices. The future work mentioned is to explore the optimization strategy of dynamically reconfigurable accelerator and improve the

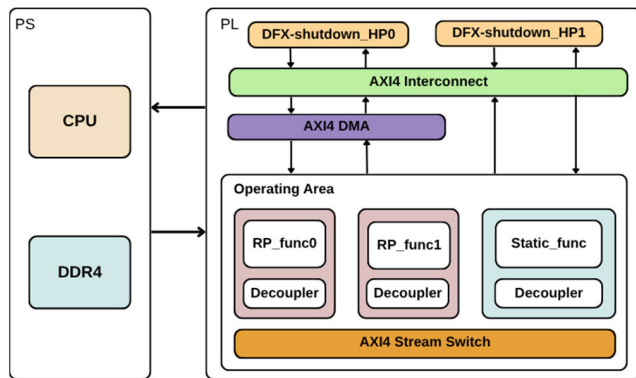


FIGURE 5. Accelerator architecture [17].

overall performance of the accelerator, making it more suitable for complex applications.

6) RECONFIGURABLE CNN ACCELERATOR FOR OBJECT DETECTION

Shi et al. propose an energy-efficient CNN accelerator for edge computing devices on an FPGA [14]. Shi introduces configuration layer sequence optimization to minimize configuration time overhead, while also implementing accelerator templates for dynamic regions to create a unified high-speed interface. This not only improves the overall performance of the system but also resolves the hardware resource constraints pertaining to traditional accelerator designs [14]. Over the past few years, research for FPGA-based CNN accelerators has focused primarily on 2 major concepts: model compression, which is crucial for deploying deep learning algorithms on resource-limited hardware, and efficient hardware architecture design [14]. Shi claims that current implementations for model compression only provide limited improvement and efficient hardware architectures tend to be divided into general-purpose and dedicated architectures. There are further areas of research to be explored such as optimization for target detection algorithms and enhancing the performance of dynamically reconfigurable hardware accelerators. This would not only minimize the initialization phase time overhead but also be applicable for industrial or even military applications [14]. Shi introduces the EDRCA hardware architecture, which is a dynamic reconfigurable accelerator. This system is deployed on the Xilinx KV260 FPGA platform. An ARM controller's processor (ARM Cortex-A7) is responsible for controlling and configuring data transmissions within the FPGA through the AXI communication bus. A dedicated accelerator template provides a uniform interface for each computation layer. The hardware platform utilized in this study shown in Figure 6 portrays the FPGA utilized in this study that provides adequate memory, frequency, and development tools for the design.

The YOLOV2-TINY model is employed utilizing the open-source VOC2012 dataset, with the convolutional



FIGURE 6. Xilinx KV260 FPGA [14].

activation layers accounting for 76.31% of the model [14]. Through experimentation, Shi found that the resources crucial for digital circuits such as Flip Flops (FF), Lookup Tables (LUT), Block Random Access Memory (BRAM), and Ultra-Random-Access Memory (URAM) had utilizations of 99.4%, 100%, 100%, and 100% respectively [14]. These impressive results portray that the proposed hardware architecture is extremely efficient, making the most use out of the hardware as possible during the duration of object detection. A different study proposed by Wang and Gu [22] to perform better in terms of computation but consume 2.14 times more power than that of Shi's design [14]. Another similar study proposed by Zhang et al. [23] developed a CNN accelerator utilizing a shrinking variable processing element (PE). This PE is responsible for performing computations in parallel in real time. While this approach was shown to obtain high computing power, it consumed more power than that of Shi. This is due to the fact that PE-Based designs require high resource utilization, which puts more strain on the physical hardware device.

7) RECONFIGURABLE CNN RTL ACCELERATOR

Kim and Choi [19] proposed an optimized CNN accelerator based on the register-transfer level (RTL). This approach achieved faster programming speed by applying low-power techniques on the FPGA. Their RTL optimization included techniques such as clock gating to eliminate residual signals, which in turn deactivates unnecessary active blocks. This reduces energy consumption by selectively deactivating inactive blocks in the hardware, directly enabling power-sensitive applications for edge computing. The proposed local explicit clock gating is shown in Figure 7. By only updating the output when EN has a logic 1, power-consumption is reduced.

This system not only saves dynamic power but also reduces processing time in the system. Various experiments were performed to validate the performance of the proposed system. They found that their system activates more register buffers which improves the performance of CNN

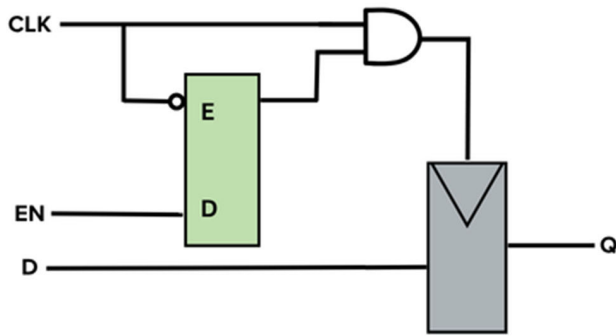


FIGURE 7. Low power clock [19].

processing. This improved processing speed is the result of high hardware resource utilization and low power consumption. They achieved 43.9 (GOPs/W) for power efficiency which is 1.37 times better than similar approaches. Additionally, the hardware resource utilization in DSPs was increased 2.2 times, this has a direct implication on the processing speed that can be achieved. 16% of the dynamic power consumption was decreased and the total On-Chip power decreased 20% of the total power consumption. The throughput result showed a 15% improvement compared with the baseline RTL code of the accelerator. The overall power consumption was reduced by 16%, and hardware utilization was increased by 58% [19]. They attained an object detection processing speed of 9.17FPS, portraying that real-time processing is feasible for mobile edge computing applications.

8) LeNet-5 CNN ACCELERATOR

Cho and Kim [13] proposed an FPGA-based CNN accelerator with a resource-optimized approximate multiply-accumulate unit. They implemented the LeNet-5 CNN model to classify handwritten digits using the MNIST handwritten digit dataset. With their proposed accelerator, operation units were implemented by utilizing an FPGAs logic resource rather than using the FPGAs digital signal processing (DSP) blocks. This was due to the fact that the DSP blocks are not as efficient for handling low-precision data in comparison to the logic resources [13]. To implement this design, they used the Xilinx XCZU9EG FPGA chip. This provides robust logic resources, adequate for the design. Their proposed accelerator consists of 3 major optimizations including loop parallelization, fixed-point optimization, and approximate MAC operations. For loop parallelization, operations in the loop body operate in parallel, promoting reduced latency and smaller execution sizes. For the fixed-point data optimization, rather than floating-point data to train the model, fixed-data points were utilized which allows the FPGA to achieve reduced power consumption, latency, and resource usage. Given the fact that the bits for fixed-data points are not shifted in any way, you can implement a more computationally efficient design with the use of fixed-data points on an FPGA. Figure 8 portrays the benefit in using fixed-data points

in comparison to floating-point. For fixed-point, unnecessary shifting of bits are eliminated.

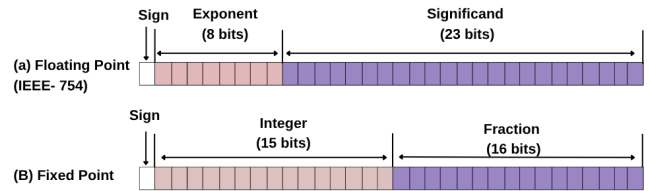


FIGURE 8. (a) Floating-Point format; (b) Fixed-Point format [13].

MAC operator modules were used for the convolutional layers within the CNN model. These MAC operations were implemented on the FPGA by using lookup tables (LUT), which proved to be very efficient given the small data size [13]. Experiments were performed to validate the accuracy and performance of the proposal. They found that their model achieved 66% less memory usage, 50% reduced network latency, and 78% fewer DSP blocks in comparison to similar designs [13]. The limitations of this design are memory bottlenecks given that they experienced issues with latency, as well as the fact that their activation functions rely on the DSP blocks of the FPGA. By eliminating the need for the DSP blocks in the activation functions of the CNN, and further reducing latency, a more robust and efficient system can be designed. One potential improvement could be to implement dynamically reconfigurable architecture that automatically adjusts based on the current available resources of the hardware. This would further promote energy efficiency and improved performance while making the design more adequate for edge computing applications.

D. REAL-WORLD APPLICATIONS

1) REAL-TIME OBJECT DETECTION AND CLASSIFICATION SYSTEM USING YOLO

Amin et al. [5] proposed a new, highly efficient FPGA-based real-time object detection and classification system using YOLOv3 Tiny for edge computing devices. This has been applied to advanced driving assistance systems for people who have visual, auditory, cognitive, or physical declines. The goal is to detect and classify traffic lights using the FPGA as an accelerant as shown in Figure 9.

The proposed system only consumes 3.5W of power, demonstrating the ability to work on edge devices (low-power systems). Accelerating deep learning models in edge devices particularly can improve the systems' performance. Using FPGAs for computer vision applications such as object detection and classification enhance real-time computing ability while reducing prices [5]. Current object detection systems require improvement amongst performance metrics such as throughput, power consumption, and accuracy, all while developing a cost-efficient design. Al Amin et al addresses this gap by developing a novel object detection system for an FPGA utilizing YOLO, achieving a tradeoff between the



FIGURE 9. FPGA accelerator for traffic light classification [5].

mentioned performance metrics. Quantization was explored to convert the 32-bit floating-point weights into INT8 fixed point values. Amin et al. compared the performance of their work with that of Heller et al. [24] given his similar usage of an FPGA and test image sizes [5]. The authors found that their work achieved a 24% improvement in accuracy, 56% reduced power consumption by hardware, and a 55% augmentation in efficiency in comparison to Heller's work. In addition, Amin et al. achieved the lowest power consumption out of 6 other proposed systems with similar criteria [5].

2) ACCELERATED FPGA-BASED PARALLEL CNN-LSTM

A parallel CNN-LSTM accelerator is proposed, in which two networks extract features from the input data synchronously, being proven to be more effective than classical CNN-LSTM implementation. Due to the different types of data that CNN and LSTM are good at processing, the requirements for input are high, which will affect the scalability of this device [25]. Since the LSTM module calculates the input data directly, there is no previous CNN pruning process compared to serial. At the same time, the gate operation of each cell of LSTM involves four activation functions and the corresponding matrix operation, so the demand for computational resources is greater when the network depth is high, and the input sequence is long. Therefore, like the CSS and handwriting classification scenarios given, the computing device designed in this paper is better at processing short time-series-based sequences. It also handles 2D data with smaller input sizes but has limited processing ability for high-depth spatial datasets or long sequence issues [25]. This is a limitation that can be addressed in future work. The future work would be to implement a fault tolerance mechanism with reconfigurable capabilities while achieving an energy-efficient system. For edge computing devices, energy efficiency is just as important as high performance. The main future developments will be finding a balance between computing performance and energy efficiency [25].

3) YOLOv5 ON Zynq-7000 SoC

Saidani et al. [26] developed a hardware acceleration platform for object detection using the YOLOv5 deep learning algorithm. Typical applications for such technology seen in industries all over the world include video surveillance, industrial design, target tracking, and Advanced Driver Assistance Systems (ADAS). A recent trend has been to design FPGAs to achieve deep learning tasks due to their reconfigurable and parallel computing capabilities. Currently, there are few studies done for FPGA-based object detection accelerators as well as limited frameworks enabling object detection designed to utilize the full capabilities of an FPGA. Saidani utilizes each layer, which is processed sequentially in hardware acceleration, of the Zynq platform to execute the YOLOv5 algorithm. Optimization of the timing of each layer's execution is explored in order while designing the streaming system. Loop pipelining is used to improve throughput in high-level synthesis due to the large amount of input data. A Leaky ReLU activation function is used in the network model to solve the gradient vanishing problem which occurs when gradients (used to update weights in the network) become very small during backpropagation, slowing the overall learning rate [20]. Using this Zynq 7020 platform, Saidani et al. achieved a maximum frequency of 250Mhz, Lookup Table (LUT) utilization of 78.99%, Flip Flop utilization of 52.76%, BRAM utilization of 46.79%, and DSP utilization of 8.64% [26]. These results demonstrate a very high resource efficiency achieving 120 frames per second (FPS) while minimizing power consumption making it very applicable for edge computing devices. A comparative analysis was performed which showed that Saidani's proposed system yielded a more efficient system, in terms of resource utilization, and reduced the time taken to perform a prediction by the algorithm. Some future considerations would be to dynamically reconfigure the architecture of the FPGA based on the current frame rate. This would not only improve the power consumption of the hardware but also improve the reliability of the system allowing for long-term operation, making it adaptable for real-time applications with varying conditions.

4) FPGA SMART CAMERA FOR SEGMENTATION

Garduño et al. [12] proposes an optimized U-Net model for segmentation applied to the Corsian fire database for wildfire detection using drones. Pruning and quantization techniques were applied to the model to reduce the number of parameters by 90%. Additionally, such techniques promote an energy-efficient design, making the most use of available hardware resources without consuming too much power. They optimized the model to improve throughput, achieving 33 FPS without loss in segmentation performance. An F1 score of 0.915 and a Hafiane quality index (HAF) of 0.70 were achieved, demonstrating an accurate and efficient approach. This model was integrated into a low-cost FPGA to implement a neural network accelerator with low power

consumption and high computational performance [12]. The authors claim there is the potential for improvement in the speed of inference by utilizing techniques such as the conversion of CNN models and spiking neural networks (SNN), which have reduced inference times. Sanchez provided a comparative analysis of his system with other similar systems proposed in recent years. Compared to the work of Akhlofi et al. [27], Sanchez achieved higher performance with his model by 2% with a 0.933 MCC and attained similar performance to the best configuration achieved by Akhloufi using a much smaller footprint.

5) INDOOR OBJECT DETECTION ACCELERATOR

Afif et al. [6] proposed a system intended for visually impaired individuals that can identify objects in indoor settings, applicable for obstacle avoidance. This work is based on a Deep Convolutional Neural Network (DCNN) which leverages a cross-stage partial network (CSPNet) for the object detection process and a lightweight backbone using EfficientNetV2 [6]. By applying the parallel computing capabilities of an FPGA, the system was optimized to reduce model size and computation complexity, which in turn, reduces the overall power consumption of the system. Such systems need to be extremely reliable and accurate as they provide these individuals with vital information that they'll need to rely on. Complex challenges of such systems include lighting conditions, different colors and shapes of the same object, and heavy occlusion [6]. Keeping this in mind, small compact systems can be heavily strained by the computing requirements for such systems. This makes the integration of FPGAs powerful parallel computing capabilities a great option for accelerating the neural network computations that would need to be performed for accurate object detection applications. Indoor navigation systems require real-time processing and need to be very lightweight for the user to easily manage [6]. Afif's proposal of leveraging an FPGA was based on a combination of CSPNet and the EfficientNetV2 networks that allow the system to achieve high accuracy, performance, low power consumption, and are applicable for edge computing applications [6]. There have been many efforts to develop navigation systems in recent years, however, most systems do not address the issue of edge computing applications and low-power designs. Afif proposed a lightweight indoor object detection system that can fit the memory size of a ZCU 102 Xilinx FPGA board [6]. The system is able to identify 25 landmark indoor objects, making this a diverse option for visually impaired navigation. Upon implementing his design, experiments were performed to determine quantitative values for the performance of his system. He compared the object detection model on the Xilinx FPGA board to a NVIDIA GTX 960 GPU. While the GPU achieved a higher average clock frequency of 2,500MHz and average speed of 35 FPS in comparison to the Xilinx FPGA which achieved an average clock frequency of 200MHz and average speed of 17 FPS, the FPGA

achieved a better power consumption and power efficiency. The GPU achieved an average power consumption of 120W, whereas the FPGA achieved an average power consumption of only 5W. Additionally, the power efficiency of the FPGA was 3.4% (FPS/Watts), whereas the GPU had a power efficiency of 0.29% (FPS/Watts). This demonstrates a high-performing system, while also developing a power-efficient design which is crucial for edge computing applications. The FPGA implementation also achieved very high detection accuracies at 80.04% for the compressed model [6]. Overall, this work demonstrates a computational, power, and performance-efficient system that allows the design of an indoor object detection navigation system for the visually impaired. Future work would be to make the FPGA dynamically reconfigurable, perhaps making the architecture reconfigurable based on the current frame rate that is being achieved. This would allow the system to improve overall power consumption given that in low frame rate scenarios, less resources of the hardware will be utilized. Additional future work would be to expand the number of objects that the model can detect. In this study, the model was trained to only identify 25 indoor objects. This can be rapidly expanded while considering the different shapes and colors of the same object to make the system more diverse and reliable.

6) EYE DETECTION ACCELERATOR

Ruiz-Beltran et al. [28] embedded an eye detection algorithm on a Zynq XCZU4EV UltraScale+ multiprocessor system-on-chip (MPSoC). This platform supports many deep learning algorithms and allows him to leverage the Deep Learning Processor Unit (DPU) to accelerate CNN operations. By leveraging the parallel computing capabilities of an FPGA, he can use the DPU to accelerate convolution operations, maximum pooling, and activation function calculations. This accelerated system will be able to perform the detections at a quicker speed, while also reducing power consumption of the system which makes this proposal applicable for edge computing, IoT, and low-weight applications [28]. Similar systems have been implemented in previous years in terms of eye and iris detection, leveraging FPGAs to accelerate CNN operations. However, Beltran claims that previous works contain many false positives in their detections which make these approaches limited and less reliable. Another major limitation of previous works is the scalability and adaptability of their approach [28]. By using a DPU, it can be easy to implement different CNN models to detect different types of objects. This makes Beltrans work very flexible and adaptable. Beltran implements his CNN in the programmable logic (PL) of the FPGA, which accelerates the design and synthesis of his trained model [28]. To further accelerate this process, the DPU was used to fetch instructions and decoders from the off-chip memory to control the operation of the computing engine [28]. The architecture of the DPU can be seen in Figure 10. The DPU fetches instructions from the scheduler in order to control the computing engine, which computes the CNN operations.

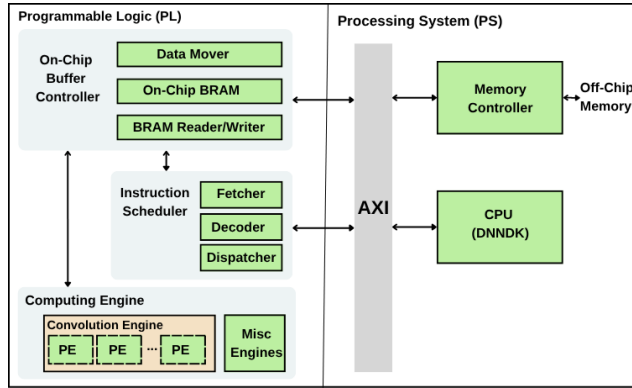


FIGURE 10. DPU architecture [28].

Upon completing the design, Beltran performed experiments to determine the accuracy and performance of his system. The proposed system achieved 100% accuracy for eye detection while achieving a throughput of 87 FPS. Such high accuracy may suggest that this proposal is too complex for the problem, which is referred to as overfitting [28]. However, Beltran claimed that these experiments were performed in controlled environments. This is significant because, in real-world applications, there are additional aspects of noise and levels of randomness that cannot be predicted. Overall, Beltran proposed very strong work that achieves extremely high accuracy and performance. By leveraging the potential of an FPGA, the system was able to be modified perfectly for the desired application and allowed for the potential to reduce power consumption for edge computing systems. Future work would be to embed the later stages of eye detection (segmentation and normalization) into the MPSoC. This would include retraining the model using images captured by the system achieving a higher performing DPU [28].

7) ERVSR ACCELERATOR

Sun et al. [29] proposed a hardware-efficient neural network for real-time video super-resolution (VSR). Their ERVSR model is built upon a recurrent CNN, which forms the basis upon which they can achieve better performance, image quality, and data throughput [29]. Their ERVSR exploits low-resolution (LR) frames, as well as information in the hidden state to develop a higher-resolution image. The ERVSR architecture retrieves low resolution frames denoted $\mathbf{x}_t$, generating a hidden state as well as a higher resolution image, denoted $\mathbf{y}_t$. The mathematical process for this architecture can be found in Equation (1) and (2):

$$h_t = NET_r(x_t, h_{t-1}) \quad (1)$$

$$y_t = NET_o(x_t, h_t) \quad (2)$$

The visual depiction of the architecture can be found in Figure 11. You can see that the NET_r block performs CNN operations on the low-resolution image, whereas NET_o takes

the output from NET_r as well as the low-resolution image to produce a higher resolution.

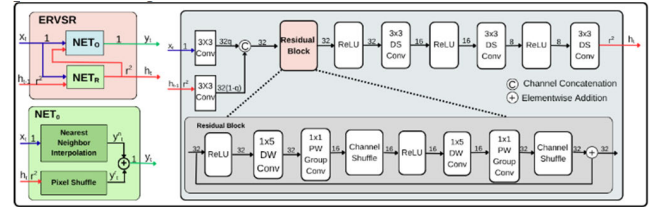


FIGURE 11. ERVSR architecture [29].

Model parameters were reduced, improving the overall efficiency of the system. Memory usage was also emphasized as they utilized the concept of quantization to reduce CNN complexity, reducing power consumption. To evaluate the effectiveness of the quantization to compress network layer size, they tested various datasets for VSR and SR. They found that their ERVSR model fewer parameters were used in comparison to similar studies. This proves that there is limited degradation in their proposed architecture. To test the temporal consistency, which demonstrates how clear the images are from the video stream, they extracted a specific region in their video frames and stacked them along the temporal dimension. They found that in comparison to similar studies, their ERVSR model has improved temporal consistency and provides clearer images. Their high-resolution video stream has the potential for improved accuracy in terms of concepts such as image segmentation or object detection. Lastly, they tested how efficient their hardware usage was. They achieved an operating frequency of 160MHz, throughput of 637Mp/Sec., power dissipation of 5.47W, and support for up to 76FPS. These impressive results indicate that Sun proposes a very hardware-efficient design that has the capability of producing high-resolution images at a high frame rate. This would be directly applicable for object detection for edge computing, as high image resolution and frame rate are crucial for accurate results while maintaining power consumption of the system. Limitations expressed include computational complexity and quantization loss. Although their ERVSR model achieved reduced model complexity, there still lies high demand of FPGA resources which can contribute not only to overall performance but also to power consumption. Improved quantization techniques can be taken to further reduce model complexity, maintaining high performance while reducing the cost of memory usage within the FPGA.

8) YOLOv2 FPGA-BASED SYSTEM

Zhang et al. [8] optimized a CNN-based object detection model to efficiently map the network by leveraging an FPGA. They developed a hardware architecture in which various convolutions use a uniform module; this approach promotes low-latency and high memory bandwidth utilization rate [8]. In this study, the YOLOv2 object detection model was improved and deployed on a Xilinx ZYNQ FPGA, which

costs around \$500. This low-cost option for improved optical remote sensing object detection provides a great approach for low-cost edge computing applications including drone, aerospace, and smart video surveillance applications. To optimize the YOLOv2 model, quantization, layer fusion, and the uniform implementation of various convolutions in the model were performed. The hardware model they designed configures the routers to perform calculations in different processing blocks on the FPGA. The fixed-point convolution results are converted to floating-point values for two fused layers; this is then followed by the conversion to integer values. A comparison operator was used to achieve max pooling in the network. Forward inference calculations consist of convolution operations, followed by further optimizations, this process can be seen in Figure 12.

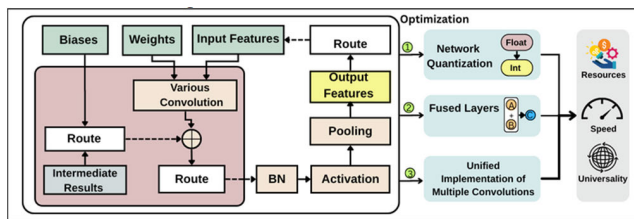


FIGURE 12. Overview of optimized implementation [8].

An experimental procedure was performed in which their FPGA implementation was compared to traditional GPU utilization for the same YOLOv2 model. They found that their approach achieved a lower performance by 0.18% in comparison to GPU results. Additionally, they found that their approach achieves a lower power consumption of 5.96W, a throughput of 111.5 GOP/s, and less than 200MHz operating frequency. Comparing these results to similar systems, Zhang's approach achieves higher energy efficiency. Future work is mentioned to improve overall performance, improve processing speed, decrease power consumption, and reduce overhead. This can be achieved by replacing external memory with the on-chip memory such as BRAM or URAM. This has the potential to improve overall performance and reduce power consumption [8].

9) FPGA-BASED YOLO SYSTEM

Pestana et al. [20] designed a configurable accelerator for YOLO object detection platforms. The proposed accelerator accelerates the execution time of the pre-processing, batch-normalization, and post-processing. Additionally, the accelerator utilizes the parallel computing capabilities of the UltraScale XCKU040 FPGA, enabling it for real-time object detection applications on edge computing devices. Given that YOLO leverage accuracy and execution time, it is a primary option for object detection applications. However, the computational requirements are very high which makes it more difficult to implement hardware-constrained devices. High computation cost is proportional to the energy-consumption of the hardware used. Therefore, Pestana deploys different

YOLO models on the FPGA to accelerate the IP core of the CNN model. Fixed-point quantization was explored to reduce storage requirements, hardware utilization, and power consumption. An experimental procedure was performed on the YOLOv3-Tiny model. They found the execution time to be 30.9ms with a throughput of 32FPS. Additionally, they found the operating frequency to be 143MHz, LUT utilization of 139, BRAM of 384, throughput of 180 GOP/s, and a power consumption of 3.87W. Comparing these results to other FPGA-based implementations, they found that their approach saw a higher LUT and BRAM utilization, as well as a higher area efficiency by 3.2 times. The future work mentioned is to improve their accelerator with additional types of layers in the CNN architecture. They acknowledge that the approach to dynamically reconfigure the YOLO model on an FPGA is resource intensive. Limiting the resources necessary to reconfigure the model can be explored to improve the performance of the system.

E. ENERGY-EFFICIENT DESIGNS

1) SparkNOC LIGHTWEIGHT ACCELERATOR

A major trend in research today is to deploy feedback, which is when input data is passed into a CNN without any feedback. However, resource-constrained devices for mobile or edge computing applications are limited in computational resources and power consumption, raising the requirement of deploying CNN feedback. Xia et al. [30] proposed a lightweight NN architecture that is capable of reducing the weight parameters of the neural network and the overall computational demands of the system. They verified the performance of their proposed design on 4 different datasets showing a compression of the CNN by 150x in each case. GPU and ASIC-based designs are a proven alternative to FPGA accelerators. However, FPGA-based designs have a significant advantage given their parallel computing, reconfigurable, and power-efficient capabilities. For applications regarding object detection or object recognition, CNNs are praised for their prominent performance. The issue is that CNNs are computationally demanding with high memory requirements. This has led to the pursuit of FPGA accelerators, which have been proven to be a viable approach. A major limitation expressed in the literature remains the fact that all operations within CNN are performed using a fixed hardware computing unit. Each layer in the CNN needs to adopt the identical computing architecture, parallelism, and reconfigurability of the FPGA. In this case, all FPGA resources cannot be fully utilized, which is inconsistent with the inherent computing characteristics of CNNs and thus causes limited parallelism and more sophisticated control flow [30]. This study prioritizes the development of a novel accelerator that can easily be deployed at the edge, mapping all the network layers to its independent customized hardware unit, and deploying the architecture on an Intel Arria 10 FPGA platform. The proposed accelerator can be found in Figure 13. The main aspect of this accelerator is the

convolution cores in which the operations for the CNN are computed.

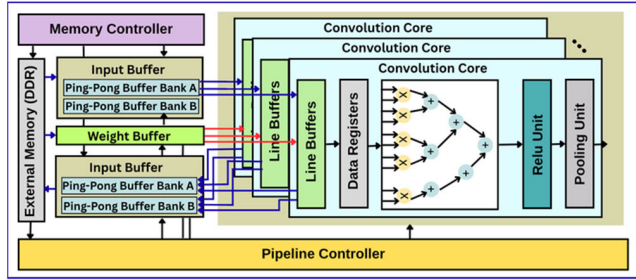


FIGURE 13. SparkNOC architecture [30].

The results obtained through experimentations show a 96.9% utilization of DSPs, 15.2% utilization of ALMs, 18.3% utilization of available registers, and 13.7% utilization of BRAM. The FPGA-based accelerator of this study is compared with other software implementations in three aspects, i.e., processing time, performance, power consumption of SparkNet. On both CPU and GPU, the average time to process 10,000 $32 \times 32 \times 3$ images is assessed with SparkNet. Since the architecture is fully pipelined, the processing time of 32×32 images is only 11.18 μ s, achieving 9 $\times$ speedup over the GPU platform and 41 $\times$ speedup over the CPU platform, respectively [30]. They also achieved a power consumption of 7.58W and a power efficiency of 44.48 GOPs/W on the Arria10 GX1150 FPGA board. Comparing this energy consumption with other FPGA boards, higher energy efficiency is prominent. Overall, this proposed work achieved high throughput and power efficiency in comparison to similar designs. The future work is to extend to larger feature maps and expand the potential edge computing applications for their design.

2) HIGH-THROUGHPUT & POWER-EFFICIENT FPGA IMPLEMENTATION OF YOLO CNN

Nguyen et al. [16] proposed a high throughput, energy-efficient, real-time object detection model by leveraging the parallel computing capabilities of an FPGA for a YOLO CNN model. They implemented the entire network model in the black RAMS of the FPGA, this eliminates the need for off-chip memory access improving performance and power consumption [16]. In the proposed system, each convolution layer in the network is pipelined to improve the utilization of available hardware resources. Intermediate data is reused across every layer in the network, this eliminates the need for external off-chip memory access, decreases DRAM access, thereby reducing power consumption of the proposed system. They designed the network such that it is capable of being scaled to any size needed for other applications. Comparing similar designs, Nguyen's proposed network architecture performs better in terms of performance achieved and power consumption [16]. They utilized a VC707 FPGA, which costs around \$5,000 to \$10,000. While this approach is not

applicable for low-cost systems, the evaluation metrics achieved are impressive and have high potential for numerous applications. The experimental procedure performed showed that their proposed CNN architecture achieved a throughput of 1.877 TOP/s, 200MHz operating frequency, with 18.29W average power consumption. There is a tradeoff between throughput achieved and power consumption of the hardware. While the 18.29W of power consumption is higher than most related works, the throughput of 1.877 TOP/s is an impressive achievement showcasing the true computational performance of the proposed system, especially for real-time object detection. The future work mentioned is to make the network more complex and deeper in order to improve the efficiency of the architecture in terms of performance.

3) RESOURCE EFFICIENT INTEGER-ARITHMETIC-ONLY FPGA-BASED CNN ACCELERATOR

Kim et al. [9] proposed a resource-efficient FPGA CNN accelerator for real-time facial emotion recognition on the Xilinx ZC706 SoC FPGA. Resource constrained embedded devices are not suitable for complex CNN architectures, making an FPGA accelerator a prime choice. Kim developed a novel quantization method, exclusively utilizing an integer-arithmetic approach, limiting model size and promoting higher performance on resource-constrained devices. Traditional quantization techniques utilize floating-point operations. Floating-point quantization tends to be more computationally demanding, eliminating its applicability to lower performing or resource constrained hardware devices. There are 4 steps for the proposed quantizer, consisting of scaling, conversion to integer values, clamping, and lastly de-quantization. This novel quantization process can be found in Figure 14.

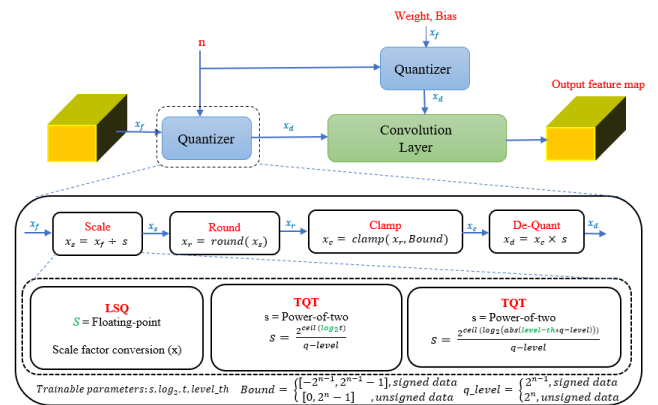


FIGURE 14. Novel quantization method [9].

They created their own dataset denoted as FERplus-A. The decision to create their own custom dataset was to improve generalization and classification achieved by the trained model. They decided to train their model on the dataset prior to applying their novel quantization approach. This was to ensure that the size of CNN parameters and number of integer operations were decreased. After quantization, they obtained

a CNN parameter size of 0.39MB, with approximately 28M integer operations needed to process a single input sample. An experimental procedure was performed to determine the classification accuracy of their trained model, in which they found the classification accuracy of facial recognition to be 86%. Comparing these results to similar approaches, they found that their approach achieved 5.07% higher accuracy, 6.33 times smaller parameter size, and larger number of operations by 1.4 times. Additionally, they achieved a clock frequency of 250MHz, with better LUT, BRAM, and DSP utilization by 4.41, 9.07, and 2.29 times respectively. The throughput, in terms of FPS, and power consumption was also observed. They achieved an average of 10 FPS with an average power consumption of 2.3W. The 10FPS they achieved was lower than that of a similar study in which they have a 20 times faster throughput with a DSP utilization 2.2 times better. Overall, Kim's proposal exhibited an accurate facial recognition model with low hardware resource utilization and low power consumption, making it applicable for edge computing applications. Future work would be to improve the FPS and improve the hardware resource utilization.

The classification carried out in this section creates an integrated picture about how different design goals such as novel hardware architectures, application-centric optimization, or energy-oriented optimization affect the design and behavior of FPGA-based CNN accelerators. This topic-based organization highlights common research directions but fails to clearly highlight technical means utilized to meet basic implementation goals. To address this challenge more effectively, Section III further organizes the designs based on four functional goals necessary for low-cost, highly efficient AI deployment at the edge: reconfigurability, power efficiency, cost, and use of hardware resources. This performance-oriented perspective enables us to extract richer insights into how design goals can impact the performance of the system and how low-cost systems are inherently lower performing. While an inexpensive system is very attractive for edge computing, they tend to struggle to meet fundamental performance metrics due to their low resource utility. The following section compares the performance metrics and system factors mentioned, noting important trends and tradeoffs. Moreover, every low-cost system examined will be compared based on an introduced scoring method assessing throughput, accuracy, power consumption, real-time applicability, and hardware efficiency. These characteristics clearly describe how well the system performs on hardware and software level. This allows us to quantify each low-cost system which allows us to easily identify the top performing low-cost accelerators and discuss opportunities for future improvement in resource-constrained edge deployments.

F. CLASSIFICATION BY IMPLEMENTATION CONSTRAINTS: RECONFIGURABILITY, EFFICIENCY, COST, AND RESOURCE UTILIZATION

Many FPGA-based CNN accelerators today share common limitations including high power consumption, high cost,

high memory consumption, and strain on the available hardware resources of the FPGA. It is crucial to develop a high-performing system at a low cost with low power consumption, particularly for edge computing applications where these factors play a vital role in its operation. Lin et al. [18] described the necessity for an optimized hardware system that utilizes only the resources necessary for that given moment in time. The authors claimed that most FPGA-based implementations today for CNN models lack factors such as dynamic reconfiguration techniques which can adjust the architecture of the system on the fly, utilizing resources that are needed based on the complexity of the captured frames. For real-time applications such as object detection, such techniques would have the potential to result in a more hardware-efficient system, thus having a lower overall power consumption while maintaining its performance.

To expand upon individual summaries of the papers and extract more general architectural insights, this section summarizes the literature surveyed by directly comparing key implementation details of 20 FPGA-based CNN accelerators. Table 2 summarizes performance metrics such as power usage, energy efficiency techniques, throughput, hardware utilization, and system expense. These categories were selected based on the prominence of each field in the literature surveyed, as well as their applicability to real-time, low-expense edge computing applications. This structured comparison enables us to compare directly, side-by-side, each system, and identify valuable tradeoffs and trends that emerge across design types.

Sub-section 2 classified designs in terms of high-level design motivation, novel architecture, application specificity, and energy-oriented approaches, this sub-section classifies the same designs into categories according to four low-level implementation goals: reconfigurability, power efficiency, cost-effectiveness, and hardware resource utilization. These goals are the most restrictive constraints for deployable systems at the edge and allow for more functionally based comparison of the behavior of accelerators reviewed. This shift in focus from design intent to practical performance and feasibility of deployment is accomplished in this chapter.

1) RECONFIGURABLE DESIGNS

There have been many dynamically reconfigurable systems proposed in which real-time adaptability of the system's architecture was explored [8], [9], [10], [11], [12]. The benefit to such an approach is the fact that the system will be able to handle unforeseen changes in conditions without ceasing computational operation. However, a common theme amongst these designs is that their systems offer a computationally complex system with limited memory bandwidth. These factors were shown to result in a system that is limited in its potential maximum throughput, as well as the latency that can be achieved. For real-time applications such as object detection where the computational demand can vary based on the complexity of the captured frame, these designs lack the ability to optimize resource utiliza-

tion of the FPGA which will limit the system's ability to perform highly. In terms of performance metrics achieved, Syed et al. [10], who proposed a system that alters the parameters of the architecture depending on detected faults achieved an accuracy of 90% with 108.88 GOP/s. The throughput achieved in this case was higher than that of Shi et al. [14] who achieved an even lower number of operations per second of 75.19 GOP/s. To account for these lower performance metrics, efforts have been made to design systems that are only partially reconfigurable. Such an approach considers flexibility, resource utilization, and performance. Irmak et al. [11] proposed a Dynamic Partial Reconfiguration (DPR) based FPGA with an accuracy of 99.3%, 7x higher frame rate, and 7-10% lower power consumption in comparison to similar approaches. However, Irmak again struggled with the number of operations attained per second at 40.7 GOP/s. Chang et al. [15] also proposed dynamically reconfigurable architecture with a high throughput of 809-1503 GOP/s but struggled with power consumption (12.6-15.3W) and had a high cost of approximately \$3,500. A dynamically reconfigurable architecture shows promise for edge computing applications, but an adequate balance between throughput, accuracy, power consumption, and cost are necessary.

2) LOW-POWER DESIGNS

In designing FPGA hardware accelerators for edge computing applications, low power consumption is a major priority. This is due to the fact that most edge computing devices often require batteries for operation, drastically limiting the amount of power that can be consumed for long-term use. There have been many efforts to design an FPGA CNN accelerator for object detection/recognition applications that have a very low power-consumption of less than 5W [2], [5], [6], [9], [10], [17], and [20]. However, this often comes at a tradeoff for performance achieved and cost of the system. Syed [10] achieved an extremely low power consumption of 0.393W-0.66W depending on the model used with a cost of approximately \$15,000. Despite this low power consumption, for low-cost edge computing applications Syed's approach is not applicable. Amin et al. [5] attained a power consumption of 3.5W with high accuracy and low cost but achieved a low throughput of 15FPS. Afif et al. [6] achieved a higher throughput than Al [5] at 17PS but had a higher power consumption of 5W. Hale [17] and Kim et al. [9] both had a lower power consumption than that of Amin's at 2.92W and 2.3W respectively but saw an even lower throughput. Pestana et al. [20] saw a slightly higher power consumption of 3.87W but had a much higher throughput of 32 FPS for the YOLOv3-Tiny model and 31 FPS for the YOLOv4-Tiny model. Pestana's higher throughput and balanced power consumption came at the price of the higher cost for his proposed system at around \$1,000.

3) LOW-COST DESIGNS

A crucial aspect in designing an FPGA CNN accelerator for edge computing applications is the cost of the FPGA platform utilized. Many studies have investigated low-cost solutions of less than \$1,000, with attempts to maximize the accuracy and performance metrics achieved [5], [8], [11], [12], [17], [19], [20], [28]. The authors of [5], [11], [12], [17], and [19] all developed their accelerator on an FPGA platform for less than \$500. For FPGA platforms between \$500 and \$1,000, there does not seem to be a significant difference in terms of the accuracy, throughput, and power consumption that can be achieved. While some studies attained a low throughput of 9.17 FPS [19], and high-power consumption of 51W [28], there does not seem to be a trend in terms of performance for FPGA platforms within this price range. Al Amin et al. [5] utilized an FPGA platform for only \$250 and attained a 99% accuracy, throughput of 15FPS, and power consumption of 3.5W. Additionally, Irmak et al. [11] utilized an FPGA platform with dynamic partially reconfigurable capabilities for around \$300-\$500, achieving a high accuracy of 99.3% and had a 7-10% lower power consumption in comparison to static designs. This proves that a highly accurate, high throughput, and low power consuming accelerators can be implemented on an FPGA platform for less than \$500.

4) HIGH RESOURCE UTILIZATION DESIGNS

In terms of hardware resource utilization, LUT, BRAM/DRAM, and DSP are vital metrics in determining how well the accelerator make use of the available FPGA resources. The hardware resource utilization is directly correlated to the throughput in terms of the number of operations per second that the system can achieve. This is evident in numerous studies where high hardware resource utilization corresponds to higher throughput [9], [16], [20], and [26]. Pestana et al. [20] had a LUT and BRAM utilization of 139 and 384 respectively with a throughput of 180 GOP/s and 32 FPS. Comparing Pestana's work to similar studies, they found a higher LUT and BRAM utilization of around 3.4 times, resulting in a higher throughput attainable. Saidani et al. [26] saw a LUT, BRAM, and DSP utilization of 78%, 42%, and 8.4% respectively resulting in a throughput of 120 FPS for his real-time object detection application. Kim et al. [9] attained 1.4 times higher operations per second in comparison to similar studies with an improved LUT, BRAM, and DSP utilization by 4.41, 9.07, and 2.29 times and a power consumption of 2.3W. Nguyen et al. [16] saw an enormous throughput of 1.877 TOP/s with a very high LUT, BRAM, and DSP utilization of. Nguyen's high throughput came as the cost of higher power consumption at 18.29W in comparison to Pestana at 3.87W and Kim at 2.3W. While there is a direct proportional correlation to hardware utilization and throughput, this tends to come at the cost of higher power consumption, thus making the system less applicable for edge computing applications.

5) SUMMARY

After comparing crucial performance metrics amongst these approaches, as well as approaches taken to implement CNN-Based accelerators, it is clear that dynamically reconfigurable architectures that are low cost with high hardware resource utilization are most desirable. There will always be trade-offs between cost, power consumption, and performance, however, attaining a balance between these metrics is more than reasonable. Both hardware and software optimization are critical for optimal designs. While model compression techniques including hardware pruning and quantization reduce power consumption and have the potential for increased throughput, accuracy can be hindered. To accommodate this, the neural network architecture needs to be stable and trained to have low loss with high accuracy. The performance of the model needs to be verified using an error function before performing compression techniques. This will not only ensure that the model is optimized for the FPGA but will also perform well after deployment in terms of accuracy. Additionally, approaches that have maximized on-chip memory including BRAM, URAM, and DSPs saw lower latency and reduced power consumption.

For a more detailed comparison amongst the proposals specified in the literature survey, we created a comparison chart specifying the differences between vital metrics including cost, power consumption, key limitations, throughput, and more. This comparison chart can be seen in Table 2. The chart provides immediate and rigorous insights into each proposal, promoting an efficient and direct comparison between reviewed approaches.

III. ANALYSIS OF LOW-COST DESIGNS

As DL models continue to be deployed to embedded or edge environments, low-cost FPGA-based accelerators are becoming a steadily more appealing alternative to GPUs and CPUs. This is especially the case in applications where power consumption, real-time responsiveness, and budget become paramount. While such affordable platforms have limited hardware resources such as fewer LUTs, FFs, and DSP slices compared to more expensive FPGAs, these platforms are still viable options since they can be optimized to deliver competitive performance-per-watt. Their energy efficiency and parallel compute capability make them particularly well-suited to run in resource-limited systems like smart cameras, autonomous sensors, and edge AI modules. To enable continued innovation and standardization within this research field, we present a benchmarking comparison of low-cost state-of-the-art FPGA-based accelerators reviewed in this paper. Each low-cost solution included in our analysis costs less than \$1,000. By emphasizing reproducibility, real-world deployability, and consistency with state-of-the-art AI benchmarking protocols, we provide insights into current design challenges for low-cost solutions, prominent tradeoffs in performance, and guidance for the future development of low-cost solutions.

To analyze the low-cost FPGA-based CNN accelerators for real-time edge computing applications, we selected the five most critical performance metrics: throughput, accuracy, power efficiency, real-time applicability, and efficiency (throughput/watt). These evaluation metrics are derived from testing methodologies of two widely utilized and prominent benchmarking environments. These benchmarks include MLPerf Tiny [31] from MLCommons for inference workloads in embedded system setups, and EdgeBench [32], an edge computing platform measurement suite for standard workloads. Collectively, these paradigms form the theoretical and application-critical significance of our target measures regarding edge deployment for low-cost FPGA-Based CNN accelerators.

Throughput, measured in terms of giga operations per second (GOP/s) or frames per second (FPS), is an indication of the rate at which an accelerator can handle received data, and is a common system performance measure within software as well as hardware applications. While throughput does not appear as an independent metric in MLPerf Tiny [31], it indirectly manages this performance aspect through inference latency and speed benchmarking on image classification. Throughput remains a significant measure of the ability of a system to process high-rate input in edge computing environments. Each approach reviewed in this paper provided throughput, either in terms of operations performed per second or in frame processed per second. This further enforces the significance of this evaluation parameter. Throughput measures the raw computational power of the system. For the development of an FPGA-Based CNN accelerators, the computational power of the system is one of the most critical aspects of the design.

The accuracy of the deployed DL model is also crucial because the inference output is degraded by performance optimizations like pruning and quantization. MLPerf Tiny [31] poses extremely minimal accuracy requirements for the validity of the model (85% image classification rate). This requirement makes accuracy a challenging parameter to employ in determining the quality and deployability of compressed or quantized models, particularly in resource-limited edge computing environments.

Power consumption is one of the fundamental limits for edge devices, which must be executed under low power budgets and thermal margin. Energy-based measurement is facilitated by MLPerf Tiny [31] utilizing the EEMBC EnergyRunner toolset that reports output in joules per inference. Additionally, EdgeBench [32] also provides power-related data in terms of CPU and memory usage profiling for edge computing loads. These approaches call for the measurement and minimization of energy consumption in embedded systems, particularly in highly-active (long operation times) and power-constrained applications.

Real-time applicability, typically in terms of latency, is also an important demand for real-time computing applications. The systems need to provide on-time outputs is critical to facilitate applications such as autonomous vehicle naviga-

TABLE 2. Summary of key performance metrics for each low-cost design.

System	Throughput (GOPs)	Throughput (FPS)	Accuracy (%)	Power Consumption (W)	Latency (ms)	Efficiency (GOPs/W)	Efficiency (FPS/W)
[5]	N/A	15	99	3.5	66.5	N/A	4.2
[8]	111.5	N/A	69.32	5.96	N/A	18.71	N/A
[11]	40.71	N/A	99.3	2.039	0.06219	19.96	N/A
[12]	N/A	15.77-33.63	91.5	5	29.7	N/A	3.15-6.73
[14]	75.1929	10.893	N/A	5.52	91.8	13.6219	1.974
[17]	26.2	N/A	N/A	2.92	0.142	8.97	N/A
[19]	63.3	9.17	N/A	1.44	109	43.9	6.37

tion, surveillance, and sensor-activated response. For video or image processing applications, latency signifies the amount of time it takes to process a single frame individually. This metric not only speaks to the performance of the accelerator in terms of computational power, but also how quickly data can be retrieved by the system. Fast data retrieval is ideal, as this reduces the latency to process a frame. MLPerf Tiny [31] directly measures inference latency in their benchmark suite, and EdgeBench [32] provides fine-grained time analysis like compute time, time-in-flight, and end-to-end latency. This timing measure is a gauge of the ability of a system to meet real-time requirements, which in real-world deployment is of utmost importance.

Lastly, system efficiency (throughput per watt of power consumed) is a metric that considers both computational performance and power consumption. While MLPerf Tiny [31] targets energy cost per inference and inference rate in isolation, throughput-per-watt can be computed from the latter. It is an optimal measure to compare multiple low-cost platforms or architectures having varying power profiles between them. It indicates how effective the system is in providing the highest computational load per energy unit, which is a necessity for high-performance edge solutions. Many papers reviewed in this study included an efficiency metric in their experimental results [5], [14], [17], [19], and [30], which further enforces this metrics significance. The equation utilized to determine the systems efficiency is shown below in Equation (3). This equation directly relates the throughput (in GOPs or FPS) to power consumption (watts):

Efficiency

$$= (\text{Throughput}(\text{GOPs or FPS})) / (\text{Power Consumption (W)}) \quad (3)$$

Our analysis of low-cost solutions reviewed in this paper is founded on metric priorities established in MLPerf Tiny [31] and EdgeBench [32] for context-appropriate and practice-hardened scoring. In addition to capturing the trade-offs most critical to low-cost FPGA design, these metrics are also consistent with industry-standard representations of real-time, resource-limited edge AI systems evaluation.

To formalize our analysis, we introduce Table 3, which consolidates all performance metrics that were reported by

the surveyed low-cost FPGA systems. Each design is analyzed in terms of its reported throughput, accuracy, power, latency, and efficiency (in GOPs/W and FPS/W where applicable). The purpose of this comparison table is not only to give quantitative readings of measured performance, but to identify tradeoffs between performance metrics and possible optimizations. Table 3 will also be utilized in determining grades for each low-cost system.

Given that not all systems reported metrics included in our performance evaluation, our scoring approach compensates for this by normalizing the metrics reported, rather than over all five. This ensures fairness in comparison, as well as favoring those with full and proper performance reports as much as possible. Each metric for a design that is available is scored on a relative scale from 0 to 1, with the highest-performing system in each category assigned a score of 1, and the others scaled proportionally to how well they compare to other systems. The total score for a system is the average of its available normalized scores across all reported categories. A system with five reported measures will get graded on five. This is the case for systems [5] and [11]. A system with only 3 measures reported will get graded on only the 3, seen for [12]. This allows us to visually discern tradeoffs among raw throughput, efficiency, and real-time responsiveness, and to visually observe systems leveraging for a particular performance metric at the cost of others.

As an example, some designs dominate in terms of throughput, but lag in energy efficiency [8] and [14], while others prioritize low power at the expense of peak throughput [11] and [17]. This is evident by simply observing Table 3. Comparing these trends, we try to bring out beneficial design trends on low-cost platforms and offer intuition in this section. Our recommendations will be regarding which trade-offs are optimal as a function of application setting. This analysis also points to the desirability of more complete, consistent reporting of performance numbers in subsequent research to support fully open benchmarking.

To enable equitable comparison of the low-cost accelerators compared in this review, raw performance values must be normalized to a common reference. Such is particularly necessary due to the variation in the way values such as throughput (GOPs or FPS), accuracy (%), and efficiency (GOPs/W and/or FPS/W) are reported. When systems

TABLE 3. Score & evaluation for low-cost designs.

System	Throughput Score	Accuracy Score	Power Score	Latency Score	Efficiency Score	Final Score
[5]	-	0.99	0.54	0.39	-	6.4
[8]	1.0	0.0	0.0	-	0.28	3.2
[11]	0.17	1.0	0.87	1.0	0.31	6.7
[12]	-	0.74	0.21	0.73	-	5.6
[14]	0.57	-	0.10	0.16	0.13	2.4
[17]	0.0	-	0.67	1.0	0.0	4.2
[19]	0.43	-	1.0	0.0	1.0	6.1

have different scales of values, they cannot be meaningfully compared without normalization. Our analysis thus takes a min-max normalization approach which is a technique that was proven effective by Ali et al. [33]. Here, they examined its efficiency in enhancing regression performance on disparate datasets. They found min-max normalization to be a uniform technique for normalizing the importance of features and ensuring equal contribution among the evaluation measures. This equation removes the domination of some of the features over others and is required while comparing features having different numeric ranges [33]. For our analysis, this is exactly the case, which makes the use of this equation highly applicable.

In order to achieve this, Equation (4) is applied for all performance values of a given system for measures whose larger values indicate improved performance (throughput, accuracy, and efficiency). $S_{i,j}$ stands for the normalized measure for the metric i for system j , based on raw performance value $V_{i,j}$. The expressions $\min(V_i)$ and $\max(V_i)$ are the minimum and maximum values of that metric in all systems. The output, a value between 0 and 1, is an indication of the level of the system's performance and satisfaction in comparison to its peers. The normalized scores are then graded on a 0–10 scale and averaged by category to produce each system's final grade of performance. Equation (4) is shown below:

$$S_{i,j} = \frac{V_{i,j} - \min(V_i)}{\max(V_i) - \min(V_i)} \quad (4)$$

This grading system maximizes systems that are leading in a particular metric to a grade of 10, while other systems are graded proportionally relative to their standing. It is the basis for our comparison and has a direct impact on the subsequent categorical grades.

For performance measures that benefit from smaller values, such as fast response time (low latency) and low power, our approach has the opposite effect of what is produced with Equation (4). For sorting systems on the basis of such measures, we get larger values indicating the systems utilize less of the concerned resources. Such systems with lower response time or lower power usage are therefore rewarded higher.

This strategy has been adopted in studies where low-value performance is quantitatively graded in a positive way. This is the case in a study conducted by Tan et al. [34]. Although the authors give a qualitative description of how their method

is pertinent, in our paper we give a more framework-based description for it. We give a more quantitatively formalized description of what matters in measuring the performance of a decomposed system that has been critically disassembled across multiple metrics. This has motivated the utilization of Equation (5) for grading a system based on their latency and power consumption achieved. Each of which are critical performance metrics in determining the effectiveness of a system for real-time edge computing applications.

Equation (5) uses $S_{i,j}$ to indicate the normalized measure i (power and delay) of the system j , as taken from the raw value we call $V_{i,j}$. For a given metric, the computation of the normalized scores makes use of the raw scores collectively, including the max and min of the raw scores, $\max(V_i)$ and $\min(V_i)$. The operation is the inverse in the sense that a low raw score will have a high normalized score and 1 is the best performing system by the way we've defined the max and min that will be used to determine the normalized score. Equation (5) is shown below:

$$S_{i,j} = \frac{\max(V_i) - V_{i,j}}{\max(V_i) - \min(V_i)} \quad (5)$$

This solution also meets latency requirement of latency-aware systems in [34]. The utilization of this equation properly compensates for the scoring of real-time, power-aware systems.

Following normalization of each system's performance through Equation (4) for measurements where higher is better (e.g., throughput, accuracy, efficiency) and Equation (5) for measurements where lower is better (e.g., latency, power), it now comes down to aggregating all these values into a single score representative of each system. It is achieved through Equation (6), which derives the mean value of all available normalized scores per system in a range from 0 to 10:

$$\text{Final Score} = \frac{1}{N_j} \sum_{i=1}^{N_j} S_{i,j} \quad (6)$$

Here, $S_{i,j}$ is the normalized performance of system j on criterion i , and N_j is the number of reported performance criteria for the respective system. Averaging only those that are available avoids bias and does not penalize non-reporting designs since it does not reward full reporting. Simultaneously, it rewards full reporting since it allows well-documented systems to report a longer performance profile.

This is also in line with best practice in composite score estimation, as advocated by Eo et al. [35], who recommend

normalized averaging as a comprehensible and justifiable technique for aggregating multi-dimensional performance data. Their method emphasizes the need to give each component measure equal weighting unless a compelling case can be made for weighting, which is accomplished here.

Effectively, Equation (6) is an enlightening and stable way to average out a system's performance on all key evaluation parameters into a single comprehensible value. It makes easy comparisons and reports to what degree a specific accelerator makes well-rounded trade-offs in power, responsiveness, and throughput of computing, those very factors which are so critical to success with real-time low-power edge computing scenarios.

Table 1 showcases the application of Equation (4) for the determination of the normalized scores on values of throughput, accuracy, and efficiency. Similarly, Equation (5) is applied for the latency and power consumption. The individual scores for a specific metric are on a scale from 0 to 1, where a grade of 1 signifies that this system achieved the highest metric in that regard. Conversely, if a system receives a score of 0 in a metric, that signifies that it was the worst out of each of the 7 systems. As mentioned, if a study did not report a metric included in our analysis it will be excluded from its final score. These cases are shown in Table 1 by using a dash “-” in a given field. This will not penalize the system's final score in a negative way, but it does give an advantage to those systems that had a recorded metric in that field. Collectively, the final scores for each system are determined using Equation (6) on a scale from 0 to 10. For the final scores, a score closer to 0 signifies that this system was performing worse overall, whereas a score closer to 10 demonstrates a more balanced system in terms of the performance it achieves.

It is notable that throughput was quantified differently in the papers we reviewed. Some measured raw computational power, giga operations per second (GOPs), while others quantified in frames per second (FPS) to capture the frame-level processing speed of the deployed model. Since these two metrics capture fundamentally different aspects of system performance, these are not directly comparable. To ensure consistency and fairness in the scoring process, we only included systems that reported GOPs-based throughput when counting normalized throughput scores. Systems that reported only FPS were not included in that category, to avoid skewing the results.

We applied the same principle to efficiency scores. Designs that only reported GOPs per watt (GOPs/W) were included. Systems that reported FPS/W were not included in that metric, to maintain consistent calculations across all.

If a system failed to report a given metric it was simply excluded from that section of the analysis. The overall scores were then computed as the means of available metrics, rather than assuming all systems had a complete set of five. This strategy avoids penalizing designs for missing data, while still promoting complete and open performance reporting. It also keeps the composite scores significant and comparable

across the range of low-cost FPGA-based CNN accelerators we compared.

The individual metrics for each performance metric, as well as the computed final scores portrayed in Table 1 highlight clear trends and limitations among the seven low-cost FPGA-based CNN accelerators that we examined. Notably, none of the systems scored higher than 6.7 out of 10. This highlights the reality that these low-cost designs still lag in achieving balanced performance metrics across all of the major performance dimensions. The top-scoring system, [11], attained its score by delivering very high accuracy (99.3%), very low latency (0.062 ms), and moderate power consumption (2.039 W). However, even this system was deficient in raw computational power and throughput, especially when compared to System [8] which achieved the highest throughput with 111.5 GOPs but was near the bottom overall due to its low accuracy (69.32%) and highest power usage in the group, at nearly 6 watts.

Observing the normalized scores in Table 1, it's interesting to see how normalization alters interpretation. For example, Systems [5] and [11] scored 99.3% and 99.0% accuracy, but their normalized scores were 1.0 and 0.99. This discrepancy may seem extremely minimal, but it is a natural result of min-max normalization, especially considering that the full range of accuracy scores is between 69.32% and 99.3%. Conversely, [11] and [17] both achieved a perfect 1.0 latency score, even though their latencies were 0.062 ms and 0.142 ms, respectively. This is since the range of total latency values has large outliers such as 109 ms, which compresses the lower-end differences when normalizing. These are not errors, just oddities that happen when values are irregularly distributed.

These types of performance of trade-offs are seen throughout the results. System [19], for instance, obtained excellent energy efficiency (43.9 GOPs/W) and minimum power consumption (1.44 W) but the worst latency (109 ms), a clear issue for any real-time application. Meanwhile, the system in [17] obtained very low latency but poor throughput and did not report any accuracy, which penalized its final score. The proposed system in [12] is another interesting example. It didn't report GOP-level throughput or efficiency, but by mentioning its latency (29.7 ms) and power consumption (5 W), it still managed to get a good final score of 5.6. That score again points out the importance of complete and consistent metric reporting. Even though our scoring method compensates for missing data by averaging only the available values, comparisons can still be misleading when systems report such different sets of performance data.

A broader trend across all seven systems is that none of them lead in all five basic categories including throughput, accuracy, power, latency, and efficiency. Each design seems to optimize one or two strengths and fall behind in others. That's to be expected under the low-cost design limitation, but it does point to a central challenge in this space: making accelerators that are truly well-rounded and can handle edge

computing workloads that need both high efficiency and performance.

Overall, these results illustrate that while there is clearly progress in the low-cost FPGA-based accelerator space, no platform has yet achieved a balanced, state-of-the-art profile. Standardized benchmarks, more complete reporting, and more design efforts that explicitly target well-rounded performance would be well-serving to the field. In the next section, we propose potential directions for the research community, including key design considerations that would allow the future development of low-cost systems to record stronger, more consistent scores on all the key metrics.

IV. DISCUSSION

After analyzing the low-cost accelerator designs, no system satisfies all performance criteria perfectly. This is clearly the case by observing the final scores in Table 1, showcasing that no system achieved a score higher than 6.7 out of 10. While some systems had very low power consumption with high accuracy, they were a tradeoff for lower hardware efficiency in terms of the number of operations that can be performed per watt of power. This was evident for multiple systems [5], [11]. No system performed well in all 5 categories, proving that the future work for this field is abundant. Each work attempted to integrate a novel feature or approach without taking into consideration how this may affect other aspects of the system. There is a clear limitation amongst not only the systems included in the analysis of low-cost designs, but across all systems reviewed in this paper. Ideally, these AI accelerators are high performing, low-power consuming devices with high efficiency and throughput. For real-time computing applications, highly efficient devices that are capable of consuming low power with a high throughput are vital. In this section, we synthesize our findings from the literature review conducted, comparison of the works, and the analysis performed on low-cost designs to aid the future development of low-cost FPGA-Based CNN accelerators. Our recommendations will provide key insights into critical design considerations and techniques that will result in a balanced, robust, system that performs very well in terms of both hardware and software components.

Considering each of the 20 approaches reviewed in this paper, we examined several high-level architectural trends. Firstly, most architectures were optimized very narrowly in a single aspect (power or latency), often disregarding the downstream consequences that this had on model performance or system efficiency overall. The proposed system in [19], for instance, saw reductions in power consumption but had the worst latency of all low-cost solutions. Conversely, the proposed system in [8] was a high-throughput system with low accuracy and high-power consumption. These are sacrifices reflecting an even deeper problem: low-cost FPGA accelerators for CNNs are built around narrowly focused goals rather than holistic, system-level design thought. System design in the future will need to focus more on co-optimized system-wide design. This includes not only

high hardware-level performance, but also high software performance. While a very challenging task, this is achievable through meticulous co-design strategies. By considering these deployment requirements at the onset of the design, the system will be more robust and balanced. A balanced system that is high-performing at the hardware and software level is the most ideal for resource-limited edge computing environments. Additionally, a balanced system that is also low-cost addresses a wider range of potential applications.

While some systems are not included in the analysis that achieved very high-performance metrics on both the hardware and software end of the design, the FPGA platform utilized in these studies were expensive, ranging between \$2,000 and \$15,000 [6], [9], [10], [16], [25], and [29]. For the future development of this field, low-cost options will allow more researchers to investigate solutions to the current limitations while broadening potential applications. High-performing low-cost options are currently very scarce, strengthening the need for more adequate solutions in this area.

Given the limited hardware resource availability for cheaper FPGA platforms, a meticulous design is necessary. High on-chip memory usage is desirable due to reduced latency times and lower power consumption. However, this type of memory is limited and if not managed correctly there can be potential bottlenecks. The most obvious solution to this issue would be to reduce model size using techniques such as hardware pruning and quantization. These techniques drastically reduce model size and will allow more utilization of on-chip memory. Additionally, potential solutions such as mixed-precision quantization where each layer in the neural network is quantized to a different degree of intensity including INT2, INT4, INT6, or INT8 could allow even more on-chip memory utilization.

The issue with quantization and hardware pruning is that by simplifying the structure of the CNN model, you can drastically reduce accuracy. This makes sense because when the model is initially trained the weights and filters are trained based on the original architecture of the model. When you apply quantization (which maps original weight values to a lower precision) or hardware pruning (which completely removes filters in convolutional layers), the architecture of the model changes. To accommodate this challenge, these algorithms should be developed to accommodate accuracy loss. Quantization-Aware Training (QAT) algorithms have been shown to preserve accuracy loss after the conversion of weights from their original precision, as shown in Nahshan et al. [36] and Chen et al. [37]. QAT is a concept in which quantized weights are simulated as the model trains based on these fake quantized weights. By training the model during simulated quantization, the algorithm can determine the best converted weight values that result in the least amount of training and validation loss. This directly corresponds to higher potential for preserved accuracy. As for hardware pruning, an iterative approach has been shown to

preserve accuracy loss in Liu et al. [38] and Zhao et al. [39]. Typically, traditional hardware pruning algorithms prune a certain percentage of filters at once. An iterative pruning algorithm removes a small number of filters, followed by training after the filters are removed. This allows the model to slowly adapt to the changes its experiencing gradually, promoting higher accuracy preservation while still reducing the size of the model. For the future development of FPGA-Based CNN accelerators, researchers should emphasize the heavy compression of the deployed CNN model by utilizing these types of sophisticated and enhanced compression algorithms. By reducing the size of the CNN model using QAT and/or iterative hardware pruning you can reduce off-chip memory access (which results in higher power consumption and latency), while preserving accuracy. This directly accommodates both the hardware and software performance of the system.

More expensive FPGAs will have access to more on-chip memory and should therefore be utilized for future designs. Another crucial avenue in future design is taking into consideration clock speed. There is a direct tradeoff between throughput and power consumption based on clock speed. A higher clock speed will result in higher throughput, but higher power consumption. Typical clock speeds utilized is 200MHz, as this provides a nice balance between throughput and power consumption. A potential design consideration for future designs would be to implement dynamic voltage frequency scaling (DVFS). This approach will allow the clock speed to be adjusted based on parameters that you can specify. Many papers discussed dynamic reconfigurability, particularly partial dynamic reconfigurability (PDR). This is a concept in which specific regions of the FPGA will have a static design, whereas other regions will have the capability to reconfigure the architecture directly on the FPGA fabric. While fully reconfigurable designs may allow for higher performance metrics and potentially higher resource utilization including LUTs, FFs, and DSPs, there is high overhead complexity that can result in higher latency as well as power consumption.

Given PDR designs do not fully reconfigure the architecture, this overhead is reduced while still getting performance improvements. By using PDR, performance can be optimized while limiting power consumption and promoting efficient use of the hardware. For real-time computing applications, such a design would be ideal and provide many advantages over traditional microcontrollers, GPU, and ASIC-based implementations.

As we close this discussion, we would like to reiterate important design considerations for edge CNN accelerators. Providing a balance among these metrics is ideal and would result in the most adequate design within this field. Although challenging, with good design choices it is reasonable for a balanced system ideal for real-time edge computing applications:

Low-Cost: allows more researchers to investigate solutions and reaches more potential applications.

High Throughput: the device should be able to process large amounts of data in a short period, especially for real time applications.

High Accuracy: this is always an important metric regardless of the platform utilized. A high accuracy will result in a more reliable system.

Efficient: the number of operations performed per watt of power consumed dictates the overall performance of the system. You want a highly efficient system for edge computing applications for reduced power consumption while maintaining performance.

Low Power-Consumption: aside from PDR, an adequate quantization method is crucial for low power consumption. Other measures such as hardware pruning and dynamic voltage scaling can be taken. Edge computing systems rely on low power consumption.

High On-Chip Memory Utilization: given the limited memory resources available on most FPGAs, especially for low-cost options, on-chip memory usage needs to be maximized. Higher BRAM and URAM usage correlates to a higher throughput with reduced latency times. Power consumption may also be limited with high on-chip memory utilization.

V. CONCLUSION

This paper presented a literature survey and comparative study of 20 FPGA-based CNN accelerators models for edge computing applications with a focus on low-cost solutions under \$1,000 budget. We categorizing system-level design methods into five key optimization domains: quantization and compression, memory usage, reconfigurability, energy efficiency, and throughput. These categories revealed common architecture trends across the literature.

To evaluate seven low-cost designs in more details, we introduced a normalized scoring framework based on real-world benchmarks such as MLPerf Tiny and EdgeBench. Our analysis revealed a consistent challenge: while many systems excel in some performance sectors, none achieved a balanced score across all five-key metrics. This underscores the inherent difficulty of delivering high-throughput, energy-efficient acceleration within the resource constraints of low-cost FPGAs. Furthermore, we observed that improvements in hardware performance often comes at the expense reduced model accuracy – a key software-level reduction.

From our findings, several technical priorities emerge. Future architectures should adopt memory-aware co-design, aligning CNN model with the memory hierarchy of the target FPGA. Precision-scalable designs that adjust bit-width on a per-layer basis, can conserve resources without significantly compromising accuracy. Achieving balanced efficiency and throughput will also require tighter integration between algorithmic compression techniques and hardware mapping. Methods such as Quantization-Aware Training (QAT) and iterative pruning showed strong potential for maintaining accuracy while reducing model size.

The co-optimization of software and hardware will be essential in near future. Embedding compression and quantization techniques directly into the hardware-aware training process offers a promising path forward. For sub-\$1,000 deployments effective architectural features include pipelined execution engines, reconfigurable logic, and scalable memory hierarchies with optimized on-chip resource reuse. These approaches together enable edge systems that meet power and latency constraints without sacrificing accuracy, paving the way for practical, high-performance AI at the edge.

REFERENCES

- [1] K. Zeng, Q. Ma, J. W. Wu, Z. Chen, T. Shen, and C. Yan, "FPGA-based accelerator for object detection: A comprehensive survey," *J. Supercomput.*, vol. 78, no. 12, pp. 14096–14136, Aug. 2022, doi: [10.1007/s11227-022-04415-5](https://doi.org/10.1007/s11227-022-04415-5).
- [2] H. Ye, "Accelerating convolutional neural networks: Exploring FPGA-based architectures and challenges," *J. Physics: Conf. Ser.*, vol. 2786, no. 1, Jun. 2024, Art. no. 012004, doi: [10.1088/1742-6596/2786/1/012004](https://doi.org/10.1088/1742-6596/2786/1/012004).
- [3] F. N. Peccia, S. Pavlitska, T. Fleck, and O. Bringmann, "Efficient edge AI: Deploying convolutional neural networks on FPGA with the gemmini accelerator," in *Proc. 27th Euromicro Conf. Digit. Syst. Design (DSD)*, Aug. 2024, pp. 418–426, doi: [10.1109/DSD64264.2024.00062](https://doi.org/10.1109/DSD64264.2024.00062).
- [4] A. G. Blaiech, K. Ben Khalifa, C. Valderrama, M. A. C. Fernandes, and M. H. Bedoui, "A survey and taxonomy of FPGA-based deep learning accelerators," *J. Syst. Archit.*, vol. 98, pp. 331–345, Sep. 2019, doi: [10.1016/j.sysarc.2019.01.007](https://doi.org/10.1016/j.sysarc.2019.01.007).
- [5] R. A. Amin, M. Hasan, V. Wiese, and R. Obermaier, "FPGA-based real-time object detection and classification system using YOLO for edge computing," *IEEE Access*, vol. 12, pp. 73268–73278, 2024, doi: [10.1109/ACCESS.2024.3404623](https://doi.org/10.1109/ACCESS.2024.3404623).
- [6] M. Afif, R. Ayachi, Y. Said, and M. Atri, "Deep embedded lightweight CNN network for indoor objects detection on FPGA," *J. Parallel Distrib. Comput.*, vol. 201, Jul. 2025, Art. no. 105085, doi: [10.1016/j.jpdc.2025.105085](https://doi.org/10.1016/j.jpdc.2025.105085).
- [7] *Edge AI Accelerators Market Size, Share & Trend Analysis Report by Processor (CPU, GPU, ASIC, FPGA), by Device (Smartphones, IoT Devices, Robots, Cameras), by End Use (Healthcare, Automotive, Retail, Manufacturing), by Region, and Segment Forecasts*. San Francisco, CA, USA: Grand View Research, 2024.
- [8] N. Zhang, X. Wei, H. Chen, and W. Liu, "FPGA implementation for CNN-based optical remote sensing object detection," *Electronics*, vol. 10, no. 3, p. 282, Jan. 2021, doi: [10.3390/electronics10030282](https://doi.org/10.3390/electronics10030282).
- [9] J. Kim, J.-K. Kang, and Y. Kim, "A resource efficient integer-arithmetic-only FPGA-based CNN accelerator for real-time facial emotion recognition," *IEEE Access*, vol. 9, pp. 104367–104381, 2021, doi: [10.1109/ACCESS.2021.3099075](https://doi.org/10.1109/ACCESS.2021.3099075).
- [10] R. T. Syed, Y. Zhao, J. Chen, M. Andjelkovic, M. Ulbricht, and M. Krstic, "FPGA implementation of a fault-tolerant fused and branched CNN accelerator with reconfigurable capabilities," *IEEE Access*, vol. 12, pp. 57847–57862, 2024, doi: [10.1109/ACCESS.2024.3392240](https://doi.org/10.1109/ACCESS.2024.3392240).
- [11] H. Irmak, F. Corradi, P. Dettner, N. Alachiotis, and D. Ziener, "A dynamic reconfigurable architecture for hybrid spiking and convolutional FPGA-based neural network designs," *J. Low Power Electron. Appl.*, vol. 11, no. 3, p. 32, 2021, doi: [10.3390/jlpea11030032](https://doi.org/10.3390/jlpea11030032).
- [12] E. Garduño, J. Ciprian-Sanchez, V. Vazquez-Garcia, M. Gonzalez-Mendoza, G. Rodriguez-Hernandez, A. Palacios-Rosas, L. Rossi-Tisson, and G. Ochoa-Ruiz, "An FPGA smart camera implementation of segmentation models for drone wildfire imagery," *Computación y Sistemas*, vol. 27, no. 4, pp. 965–977, Dec. 2023, doi: [10.13053/cys-27-4-4773](https://doi.org/10.13053/cys-27-4-4773).
- [13] M. Cho and Y. Kim, "FPGA-based convolutional neural network accelerator with resource-optimized approximate multiply-accumulate unit," *Electronics*, vol. 10, no. 22, p. 2859, Nov. 2021, doi: [10.3390/electronics10222859](https://doi.org/10.3390/electronics10222859).
- [14] K. Shi, M. Wang, X. Tan, Q. Li, and T. Lei, "Efficient dynamic reconfigurable CNN accelerator for edge intelligence computing on FPGA," *Information*, vol. 14, no. 3, p. 194, Mar. 2023, doi: [10.3390/info14030194](https://doi.org/10.3390/info14030194).
- [15] L. Chang, S. Zhang, H. Du, Y. Chen, and S. Wang, "A reconfigurable neural network processor with tile-grained multicore pipeline for object detection on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 11, pp. 1967–1980, Nov. 2021, doi: [10.1109/TVLSI.2021.3109580](https://doi.org/10.1109/TVLSI.2021.3109580).
- [16] D. T. Nguyen, T. N. Nguyen, H. Kim, and H.-J. Lee, "A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 8, pp. 1861–1873, Aug. 2019, doi: [10.1109/TVLSI.2019.2905242](https://doi.org/10.1109/TVLSI.2019.2905242).
- [17] C. Hale, "Dynamic reconfigurable CNN accelerator for embedded edge computing: A hardware–software co-design approach to minimize power and resource consumption," *Trans. Comput. Sci. Methods*, vol. 4, no. 9, pp. 1–11, 2024. [Online]. Available: <https://pspress.org/index.php/tcsms>
- [18] W. Lin, Y. Zhu, and T. Arslan, "A dynamically reconfigurable column streaming-based convolution engine for edge AI accelerators," in *Proc. 29th IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Oct. 2022, pp. 1–4, doi: [10.1109/ICECS202256217.2022.9970898](https://doi.org/10.1109/ICECS202256217.2022.9970898).
- [19] V. H. Kim and K. K. Choi, "A reconfigurable CNN-based accelerator design for fast and energy-efficient object detection system on mobile FPGA," *IEEE Access*, vol. 11, pp. 59438–59445, 2023, doi: [10.1109/ACCESS.2023.3285279](https://doi.org/10.1109/ACCESS.2023.3285279).
- [20] D. Pestana, P. R. Miranda, J. D. Lopes, R. P. Duarte, M. P. Véstias, H. C. Neto, and J. T. De Sousa, "A full featured configurable accelerator for object detection with YOLO," *IEEE Access*, vol. 9, pp. 75864–75877, 2021, doi: [10.1109/ACCESS.2021.3081818](https://doi.org/10.1109/ACCESS.2021.3081818).
- [21] L. Luo, W. Kang, J. Liu, H. Zhang, Y. Zhang, D. Liu, and P. Ouyang, "A reconfigurable spatial architecture for energy-efficient inception neural networks," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 7–20, Mar. 2023, doi: [10.1109/JETCAS.2023.3243619](https://doi.org/10.1109/JETCAS.2023.3243619).
- [22] J. Wang and S. Gu, "FPGA implementation of object detection accelerator based on vitis-AI," in *Proc. 11th Int. Conf. Inf. Sci. Technol. (ICIST)*, May 2021, pp. 571–577, doi: [10.1109/ICIST52614.2021.9440554](https://doi.org/10.1109/ICIST52614.2021.9440554).
- [23] C. Zhang, Z. Fang, P. Zhou, P. Pan, and J. Cong, "Caffeine: Towards uniformed representation and acceleration for deep convolutional neural networks," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, Nov. 2016, pp. 1–8, doi: [10.1145/2966986.2967011](https://doi.org/10.1145/2966986.2967011).
- [24] D. Heller, M. Rizk, R. Douguet, A. Baghdadi, and J.-P. Diguët, "Marine objects detection using deep learning on embedded edge devices," in *Proc. IEEE Int. Workshop Rapid Syst. Prototyping (RSP)*, Oct. 2022, pp. 1–7, doi: [10.1109/rsp57251.2022.10039025](https://doi.org/10.1109/rsp57251.2022.10039025).
- [25] X. Zhou, W. Xie, H. Zhou, Y. Cheng, X. Wang, Y. Ren, S. Yuan, and L. Li, "An accelerated FPGA-based parallel CNN-LSTM computing device," *IEEE Access*, vol. 12, pp. 106579–106592, 2024, doi: [10.1109/ACCESS.2024.3437663](https://doi.org/10.1109/ACCESS.2024.3437663).
- [26] T. Saidani, R. Ghodhban, A. Alhomoud, A. Alshammari, H. Zayani, and M. B. Ammar, "Hardware acceleration for object detection using YOLOv5 deep learning algorithm on Xilinx Zynq FPGA platform," *Eng., Technol. Appl. Sci. Res.*, vol. 14, no. 1, pp. 13066–13071, 2024, doi: [10.48084/etasr.6761](https://doi.org/10.48084/etasr.6761).
- [27] M. A. Akhloufi, R. B. Tokime, and H. Ellassady, "Wildland fires detection and segmentation using deep learning," *Proc. SPIE*, vol. 10649, Feb. 2018, Art. no. 106490, doi: [10.1117/12.2304936](https://doi.org/10.1117/12.2304936).
- [28] C. A. Ruiz-Beltrán, A. Romero-Garcés, M. González-García, R. Marfil, and A. Bandera, "FPGA-based CNN for eye detection in an iris recognition at a distance system," *Electronics*, vol. 12, no. 22, p. 4713, Nov. 2023, doi: [10.3390/electronics12224713](https://doi.org/10.3390/electronics12224713).
- [29] K. Sun, M. Koch, Z. Wang, S. Jovanovic, H. Rabah, and S. Simon, "An FPGA-based residual recurrent neural network for real-time video super-resolution," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 4, pp. 1739–1750, Apr. 2022, doi: [10.1109/TCSVT.2021.3080241](https://doi.org/10.1109/TCSVT.2021.3080241).
- [30] M. Xia, Z. Huang, L. Tian, H. Wang, V. Chang, Y. Zhu, and S. Feng, "SparkNoC: An energy-efficiency FPGA-based accelerator using optimized lightweight CNN for edge computing," *J. Syst. Archit.*, vol. 115, May 2021, Art. no. 101991, doi: [10.1016/j.sysarc.2021.101991](https://doi.org/10.1016/j.sysarc.2021.101991).
- [31] C. Banbury et al., "MLPerf tiny benchmark," 2021, *arXiv:2106.07597*.
- [32] A. Das, S. Patterson, and M. Wittie, "EdgeBench: Benchmarking edge computing platforms," in *Proc. IEEE/ACM Int. Conf. Utility Cloud Comput. Companion (UCC Companion)*, Dec. 2018, pp. 175–180, doi: [10.1109/UCC-Companion.2018.00053](https://doi.org/10.1109/UCC-Companion.2018.00053).
- [33] P. J. Muhammad Ali, "Investigating the impact of min-max data normalization on the regression performance of K-nearest neighbor with different similarity measurements," *ARO-THE Sci. J. KOYA Univ.*, vol. 10, no. 1, pp. 85–91, Jun. 2022, doi: [10.14500/aro.10955](https://doi.org/10.14500/aro.10955).

- [34] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 2815–2823, doi: [10.1109/CVPR.2019.00293](https://doi.org/10.1109/CVPR.2019.00293).
- [35] S. Eo, H. Park, and Y. R. Kim, "Predictive & representative composite score," *Qual. Quantity*, vol. 57, no. 1, pp. 1–14, Feb. 2023, doi: [10.1007/s11135-022-01345-5](https://doi.org/10.1007/s11135-022-01345-5).
- [36] Y. Nahshan, B. Chmiel, C. Baskin, E. Zheltonozhskii, R. Banner, A. M. Bronstein, and A. Mendelson, "Loss aware post-training quantization," *Mach. Learn.*, vol. 110, nos. 11–12, pp. 3245–3262, Dec. 2021, doi: [10.1007/s10994-021-06053-z](https://doi.org/10.1007/s10994-021-06053-z).
- [37] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, and P. Luo, "EfficientQAT: Efficient quantization-aware training for large language models," 2024, *arXiv:2407.11062*.
- [38] S. Liu, Y. Cheng, F. Zha, W. Guo, L. Sun, Z. Bing, and C. Yang, "Learning effective pruning at initialization from iterative pruning," 2024, *arXiv:2408.14757*.
- [39] K. Zhao, A. Jain, and M. Zhao, "Iterative activation-based structured pruning," 2022, *arXiv:2201.09881*.



JOHN S. FATA (Member, IEEE) received the bachelor's degree in computer engineering and the master's degree in electrical engineering from Manhattan University, in 2024 and 2025, respectively. He has received two summer internships during his time of study, the first internship as an Electrical Engineering Intern at Central Hudson, in Summer 2023, and the other internship as a Research and Development Engineer at the Global Equipment Development Department, PepsiCo, Valhalla, NY, USA. At the time of this research, he was with the Department of Electrical and Computer Engineering, Manhattan University, Riverdale, NY, USA, and he is currently a Research and Development Engineer at PepsiCo on the Equipment Innovation Team. His research interests include assistive technologies for the visually impaired, neural networks, and efficient analog and digital hardware design. He was inducted into numerous honors societies during his time of study, including Tau Beta Pi as a Junior, which required him to be within the Top 12.5% of the Engineering Class, and Epsilon Sigma Pi as a Senior, which is the Highest Scholastic Honor that can be earned at Manhattan University. Additionally, he was elected Next in Merit for the Computer Engineering Medal, in 2024.



WAFI M. ELMANNAI (Member, IEEE) received the Ph.D. degree in computer science and engineering from the University of Bridgeport (UB). She is currently an Associate Professor of computer science and engineering with Manhattan University. She is also the Founder and the Co-Director of the Women's Research Mentorship Program, Manhattan University, established through the Women and Gender Resource Center and supported by the 2023–2024 AAUW Community Action Grant. This initiative promotes research engagement among undergraduate women through mentorship and academic exploration. She has supervised and co-supervised numerous undergraduate and graduate research projects. She has published extensively in respected national and international venues and has served in various professional roles, including the Institute of Electrical and Electronics Engineers (IEEE), the International Wireless Communications and Mobile Computing Conference, and American Society for Engineering Education (ASEE). She has contributed to the

peer-review process and the development of high-quality technical programs in areas, such as artificial intelligence, assistive technologies, and smart systems. Her research interests include the intersection of emerging technologies and inclusive computing, encompassing assistive technologies, smart systems, artificial intelligence, the Internet of Things (IoT), mobile computing, cybersecurity, computer architecture, and quantum cryptography.

Dr. Elmannai is a Universal Design for Learning Teaching Fellow. She is an Active Member of several professional and honor societies, including IEEE, the IEEE Communications Society, Eta Kappa Nu, Phi Kappa Phi, UPE, SWE, and Arab American Association of Engineers and Architects (AAAEA). She was a recipient of the AAUW International Fellowship. She served as the Chapter President and the Vice President of the Upsilon Pi Epsilon (UPE) Honor Society, UB AAUW Ambassador, and the Vice President of the Society of Women Engineers (SWE), UB, during her graduate studies. She continues this leadership as the IEEE-HKN Chapter Advisor at Manhattan University.



KHALED ELLEITHY (Senior Member, IEEE) has worked in academia for the past 30 years in various administrative and teaching roles, including the Ph.D. Program Director, the Online M.S. Program Advisor, the Associate Dean for Engineering, the Associate Vice President for Graduate Studies and Research, the Associate Dean of the College of Engineering, Business, and Education, and the Dean of the College of Engineering, Business, and Education. He has served as the PI or the Co-PI for over three million \$ of funded research projects in the past 20 years. Sponsors include NSF, ARDEC, the United Nations, the Connecticut NASA Space Grant, Cisco, the University of Connecticut's START Program, the University of Bridgeport's CTNEXT, Saudi Aramco, and the King Abdulaziz City for Science and Technology (KACST). He has published over 400 research papers in national and international journals and conferences, with more than 6,500 Google Scholar citations. His research interests include cybersecurity, wireless communication security, microstrip antenna design, steganography, mobile collaborative learning environments, QoS routing protocols in wireless multimedia sensor networks, disaster recovery processes over wireless sensor networks, and quantum cryptography, all making significant contributions to science and technology.

Dr. Elleithy was elected as a fellow of African Academy of Sciences in recognition of his contributions to wireless sensor networks and wireless communications, in December 2017. In 2020, he was honored as the IEEE Connecticut Section Outstanding Member in Academia. He has actively participated in many professional societies over the past 30 years, including the Institute of Electrical and Electronics Engineers (IEEE), the Association for Computing Machinery (ACM), and American Society for Engineering Education (ASEE). This participation includes organizing conferences and workshops, taking on leadership roles, editing journals, and engaging in various other activities. He received the Distinguished Professor of the Year Award from the University of Bridgeport, in 2005. He was awarded the 2015 Connecticut Quality Improvement Award (CQIA) Gold Innovation Award.

...